

O'REILLY®

Early
Release

RAW &
UNEDITED



Hands-On LLM Serving and Optimization

Hosting LLMs at Scale

Chi Wang & Peiheng Hu

Hands-On LLM Serving and Optimization

Hosting LLMs at Scale

With Early Release ebooks, you get books in their earliest form—the authors' raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Chi Wang and Peiheng Hu

O'REILLY®

Hands-On LLM Serving and Optimization

by Chi Wang and Peiheng Hu

Copyright © 2025 Chi wang and Peiheng Hu. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Acquisitions Editor: Nicole Butterfield
- Development Editor: Sarah Grey
- Production Editor: Ashley Stussy
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery

- Illustrator: Kate Dullea
- April 2026: First Edition

Revision History for the Early Release

- 2025-05-05: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341621497> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. Hands-On LLM Serving and Optimization, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property

rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-62144-2

[FILL IN]

Brief Table of Contents (*Not Yet Final*)

Chapter 1: Introduction to Model Serving and Optimization (available)

Chapter 2: Large Language Model (LLM) Serving (unavailable)

Chapter 3: Model Serving Best Practices and Case Studies (unavailable)

Chapter 4: Build an Agent Application with LLM from Scratch
(unavailable)

Chapter 5: Performance Challenges When Serving LLMs (available)

Chapter 6: Common Optimization Techniques (unavailable)

Chapter 7: LLM-Specific Optimization Techniques (unavailable)

Chapter 8: LLM Serving Frameworks Overview (unavailable)

Chapter 9: LLM Serving Optimization Best Practices and Case Study
(unavailable)

Chapter 10: Efficiently Serve Multiple Fine-Tuned LLM Models
(unavailable)

Chapter 1. Introduction to Model Serving and Optimization

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sgrey@oreilly.com.

At its core, *model serving* is a process that addresses the challenge of making AI models accessible to end-users, applications, and systems, working through APIs, web services, or integrated workflows to generate predictions (called *inferences*) on new, unseen data.

To draw a simple analogy, to businesses of all kinds—whether they aim to deliver AI capabilities to their customers or enhance operational efficiency—model serving is a form of supply chain. For example, Amazon and Netflix use model serving to update customer recommendations instantly as

users browse. Banks use model serving to block fraudulent transactions during online shopping checkouts, and airline chatbots use it to provide instant flight updates and rebooking options. Business stops for these companies if their model serving systems go down.

Just as every manufacturer strives for an efficient and cost-effective supply chain, AI companies seek to utilize their models and hardware efficiently and effectively in production environments. Therefore, choosing the right model serving approach and optimizing it are critical: it directly impacts the business's lifeline and operational costs. Regardless of what role you have, if you are in the AI industry, you would benefit from having some knowledge of model serving.

As tech leaders who have worked on model serving infrastructure for over a decade, we have collaborated with a variety of participants in the field: researchers, developers, executives, marketing, customers, and students. I have found that people often feel intimidated or overwhelmed when they first attempt to understand model serving. There are three main reasons for this. First, you should already have a deep knowledge of model training. Second, as yet, there's been no clear learning path that goes from get-started tutorials to managing a world-class serving system. Third, there are so many frameworks, libraries, vendors, and other engineering options that it can be hard to make decisions about which to adopt. This book aims to address the above challenges by providing a structured, practical guide to

model serving and optimization—one that bridges the gap between theory and practice while empowering readers to make informed decisions.

In this chapter, we lay the foundation for you to understand the rest of the book. We begin by clarifying the core concepts of model serving, followed by a discussion of why robust, optimized model serving is critical for real-world applications. Finally, we explore general paradigms in model serving. By the end of this chapter, you will have a comprehensive overview of model serving and optimization, setting the stage for the hands-on materials in the subsequent chapters.

What Is a Model?

In academic terms, a *machine-learning (ML) model* is a mathematical representation or algorithm that learns patterns from data to make predictions, decisions, or inferences without being explicitly programmed for the task.

In engineering and operations, we focus more on how to *use* models than how to train them. Therefore we simply treat models in black-box fashion – as collections of (executable) files produced by ML training processes.

Model

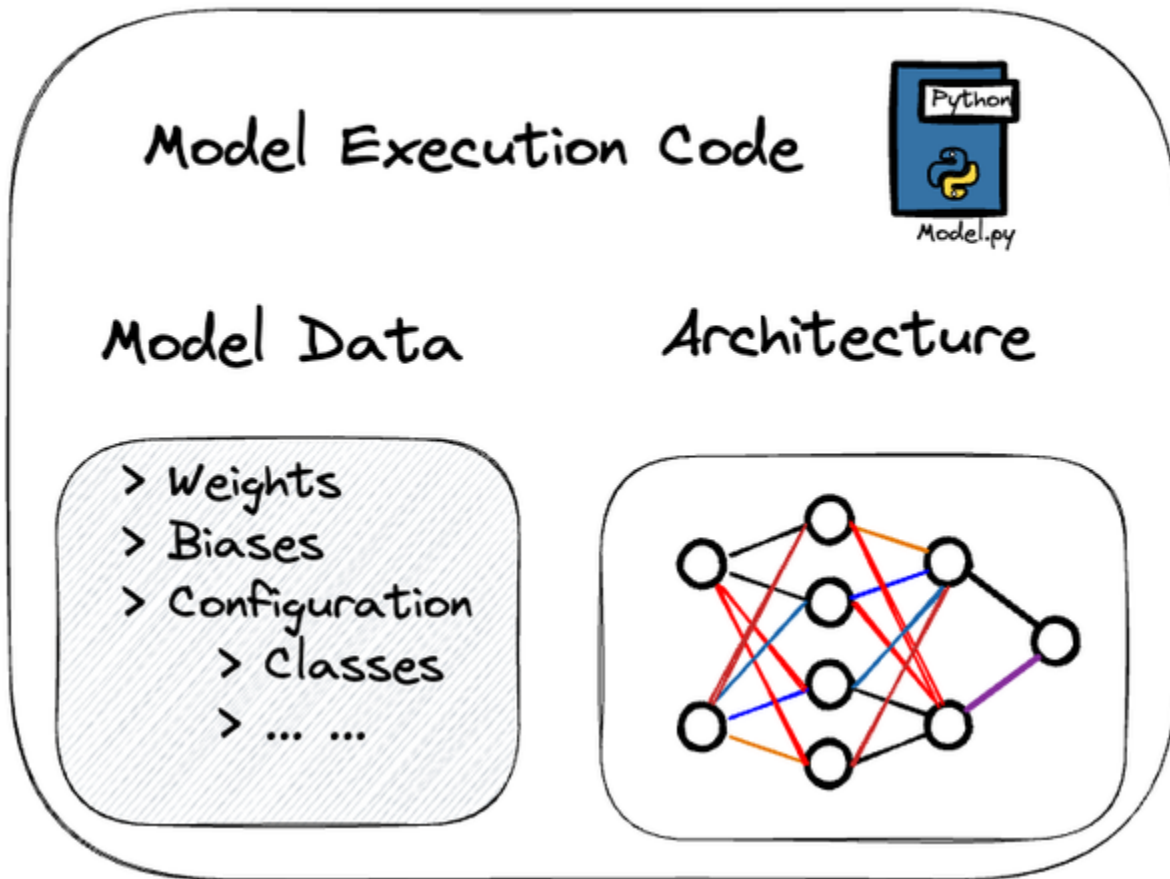


Figure 1-1. A model is composed of model architecture, model execution code and model data.

We view models as composed of three types of files: data, architecture, and execution code (see [Figure 1-1](#)):

Model data

A model's data includes its weights, bias, and configuration. Weights and bias are what the model learns during training, and the model configuration holds the metadata to run the model, such as its

embeddings and label classes (for classification models), its `max_batch_size` property (for batch inference), and its input and output tensors.

Model architecture

Architecture refers to the structure and design of an ML model. It defines how the model is organized, including the types and number of layers, the connections between layers, and the operations the model performs. The architecture determines how the model processes input data to produce output predictions or decisions.

Model execution code

A model's *execution code* is what the model runs. It generally initializes the architecture in the model serving framework, loads weights, and runs predictions (or other outputs).

TIP

You can think of models as executable program files. Many people mistakenly assume that models are merely passive data files, but in reality, they also include execution logic, metadata, and other components that define how they operate. Rather than being static artifacts, models function as dynamic programs that can process inputs, make decisions, and evolve over time.

Let's examine the structure of the model from [Figure 1-1](#) using an example from the [PyTorch](#) tutorial – "[Saving and Loading Models](#)" – which saves a

trained model to local drive.

Model Architecture

First, the following Python file defines the model architecture. This code (located in the model file/package) is normally a copy of the model architecture class in the model training code.

```
class TheModelClass(nn.Module):
    def __init__(self):
        super(TheModelClass, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Model Data

Second, after model training, the weights and biases are saved as a file called *model_weights.pt*.

```
// In PyTorch, the learnable parameters (i.e. weights and biases)
// are stored in "state_dict"
torch.save(model.state_dict(), model_weights.pt)
```

Model Execution Code

Third, the following example code defines how to execute the model for a given input:

```
model = TheModelClass(*args, **kwargs) //Initialize model
model.load_state_dict(torch.load("model_weights.pt")) //Load model weights ("model_weights.pt")
model.eval() // set model to evaluation mode to avoid dropout
pred = model(inputs) // run model prediction with given inputs
```

Although different machine learning (ML) frameworks, such as TensorFlow and PyTorch, provide distinct APIs and libraries, the fundamental principles of model packaging and structure ([Figure 1-1](#)) remain largely similar. In practice, additional optimizations are often applied to model files to

enhance performance for model serving, particularly for large language models (LLMs). These optimizations will be discussed in Chapters 6 and 7.

WE RECOMMEND STORING YOUR MODEL'S ARCHITECTURE AND DATA IN SEPARATE FILES

While it is possible to save an entire model in a single file (for example, `torch.save(model, "model.pt")`), separating the class definition (architecture) from the weights provides greater flexibility. This approach allows for more complex serving scenarios, such as loading pretrained weights into an updated model architecture while ignoring nonmatching keys or partially loading a model.

Now that you understand what an ML model is, let's move on to model serving.

What Is Model Serving?

In engineering terminology, *model serving* refers to deploying an ML model in a production environment, where it can process new data and generate predictions.¹ This involves setting up the necessary infrastructure—both software and hardware—to ensure that the model can receive input, execute inference, and return results efficiently, scalably, and reliably.

Model serving can happen *locally* (on-device), *in-cluster* (on-premises), or *remotely* (on-cloud), depending on the business requirements and

infrastructure constraints. Here are three examples, one for each serving scenario:

Warehouse robot (on-device)

A real-time object detection model (such as YOLO) is deployed on a robot's onboard computer (say, a Raspberry Pi with a Coral TPU). This lets the robot process visual input and make autonomous decisions instantly.

Document search (on-premises)

To build a semantic search index on internal company data, a language model (like Sentence-BERT) is served within the company's computing cluster. This setup generates dense vector embeddings for documents while ensuring data security and compliance by avoiding large-scale data transfers.

Customer support chatbot (on-cloud)

When a customer submits a query, the chatbot sends it, along with relevant documents, to a remote LLM hosted or provided by a vendor, such as [Amazon SageMaker Inference](#) or [OpenAI](#), as shown in [Figure 1-2](#).

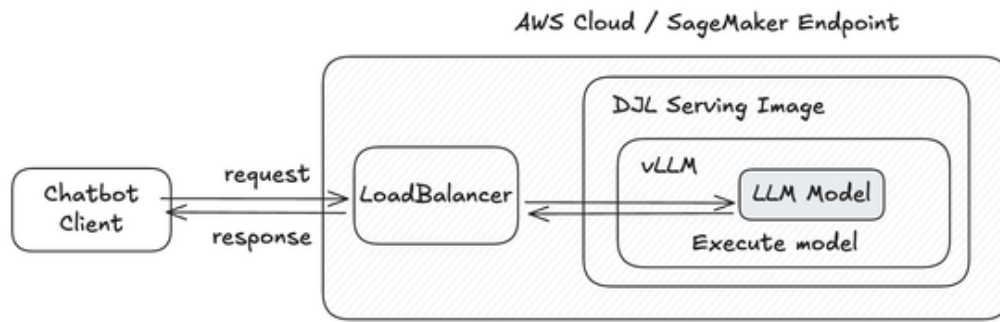


Figure 1-2. Serving a LLM model for a chatbot app in SageMaker Inference Endpoint.

In the model-serving context, ML and DevOps engineers typically focus less on models' internal details—such as their architecture, training methods, and file formats—and instead *treat models as black boxes*. This is because modern model-serving frameworks like vLLM and TensorRT-LLM handle the complexities of model execution for us, providing a high-level abstraction that makes running model predictions simpler and more efficient.

In model serving, we focus more on the following:

Deployment

Choosing the right hardware and making the model available from the training pipeline (or from open source options) to consume input data and return predictions.

Scalability and Availability

The ability to handle a few thousand to millions of requests efficiently while ensuring a consistent customer experience.

Latency

Delivering predictions quickly—in milliseconds, for real-time use cases.

Monitoring

Tracking model performance, data drift, and system health.

Versioning

Managing model updates and rollbacks without disrupting clients.

Security

Sensitive data must be protected, and access to the model should be controlled.

Cost to Serve

The most decisive factor of them all. We use cost-to-serve as a key factor to evaluate different serving approaches and trade-offs.

Model serving is a highly practical, engineering-focused field. Unlike AI research and model training, it does not require a deep understanding of ML algorithms or an academic background in AI. Instead, it emphasizes

deploying and integrating models into real-world applications using existing tools and frameworks.

NOTE

For effective LLM serving and optimization, a solid understanding of AI algorithms, particularly [Transformers](#), is essential. All advancements in LLM serving techniques are designed to overcome bottlenecks in LLM execution. So a strong grasp of LLM architecture provides us the intuition needed to optimize throughput and latency. In the following chapters, we will cover the foundational knowledge to help you understand the LLM serving and optimization methods.

Why Study Model Serving?

Most people would agree that model serving is critical, but aren't sure why they need to learn to build or customize their own model-serving systems. In this section, we address some of these questions with our own experiences.

Question 1: “We consume models directly from cloud vendors such as Amazon AWS (SageMaker or BedRock) and Microsoft Azure ML. Why would we need to learn to build a model serving system?”

Deploying and hosting models on cloud vendors' platforms is a good option in many situations. Cloud services can take care of many things, such as hardware management and software patching, and enable quick proofs of concept (POCs). We have successfully designed and built lots of model

serving solutions on AWS and Azure . However, you can never simply consume these services without careful consideration.

While cloud-based model-serving solutions are convenient, they also come with challenges. First, there are numerous products and features to choose from, making it difficult to determine the best fit. Second, integrating vendor services with existing infrastructure and client applications often requires additional engineering efforts. Third, you'll often need to optimize and fine-tune vendor services to align with your specific business use cases and performance requirements.

Cloud vendors design their model-serving solutions based on various customer needs and package them as general-purpose offerings. For instance, AWS SageMaker provides multiple serving options, including single-model and multi-model endpoints, and each option comes with different levels of customization, trade-offs, and costs. Without a clear understanding of model serving design, challenges, and use cases, it's easy to make suboptimal choices that may increase costs or limit flexibility.

In short, understanding how a model serving system is built and operated helps you make the best use of cloud vendors' model serving options.

Question 2: “Foundation LLMs are so powerful that we don’t need to consume any other models. We just let our client application call

foundation model vendors, such as Open AI, DeepSeek, and Anthropic. Why would we need to build and maintain a model serving stack?”

LLMs are one of the greatest inventions in the AI field and a milestone on the path toward artificial general intelligence (AGI). But there is a lot of nuance when you actually apply it in business.

First, there are cost considerations. Businesses hardly ever use a single LLM to solve all of their business use cases. In the real world, businesses prefer to use smaller and cheaper models as much as possible and leverage LLMs for complicated requests. Also, when usage picks up, they generally want to try adopting lower-cost LLM serving. For example, running an open source LLM (such as DeepSeek) on an AWS Elastic Compute Cloud (EC2) instance (that is, a virtual server) can be 30 to 60% cheaper than doing so on a managed serving solution like AWS SageMaker. Also, you can often get a good deal (say, a 70% discount) if you reserve an EC2 instance, which further reduces the serving cost.

A typical chatbot app might use a smaller intent-classification model to triage customer questions, an embedding model to encode those customer questions and search for potential answers, and an LLM to break the customer’s request down into actions (backend API calls) and generate human-friendly user interactions.

Another aspect is data privacy and security. If your company is handling sensitive or proprietary data, you might be forced to leverage an in-house solution to avoid data breaches or compliance-related issues.

Lastly, most foundation-model vendors either don't provide the functionality to serve your fine-tuned model, or charge a much higher price if you need to serve your own fine-tuned version of the model. For example, as we write this book in early 2025, OpenAI charges 50% more to serve a fine-tuned model. At the same time, smaller models are becoming much more powerful and providing fine-tuning for custom data and requirements, faster inference, better and more consistent results, and a lower cost-to-serve.

As technology keeps developing, it trends toward affordable and open approaches, and LLM businesses are no exception. There is nothing wrong with consuming LLMs from vendors, but having your own serving solution (on cloud or on-premises) with an open source LLM will give your business a critical edge at a lower cost.

Question 3: “Building and maintaining our own serving stack is expensive. What would make it worth the cost?”

Reducing model serving costs is an everlasting effort. As the industry develops and technology and infrastructure change, no one can guarantee that simply outsourcing your serving component will always meet your

business's goals. Understanding model serving deeply can help you understand all the trade-offs in order to make optimal choices. Let's look at a few examples.

First, imagine you run a Series A startup in the education sector. Your app leverages LLMs to grade students' homework and generate personalized study plans. Working with a fully managed LLM provider like OpenAI, DeepSeek, or Anthropic, is a great starting point to accelerate development and validate your business model.

However, as your usage scales, the serving costs start to become unsustainable. To optimize expenses, you consider migrating to a customized model serving solution: for example, using open-source LLMs, selecting optimized serving frameworks, and fine-tuning inference performance. This should significantly reduce your costs while maintaining flexibility and keeping you in control.

Let's look at another scenario. This time, you run a customer relationship management (CRM) company that leverages LLMs to provide sales recommendations by processing data from multiple sources, including text-based inputs (emails, Slack messages, customer notes), unstructured audio searches (voice calls, meeting transcriptions) and structured CRM databases (past sales records, customer interactions).

Given the high volume of LLM inference requests and the involvement of sensitive customer data, fully outsourcing your model serving could introduce data-security risks and escalate your operational costs. You decide that owning and optimizing an in-house serving stack is necessary to lower the cost per inference, comply with security regulations (like GDPR and HIPAA), and achieve better performance and latency control.

Our experience has been that while outsourced solutions can be a great starting point, businesses must continuously assess whether in-house serving provides a long-term competitive advantage in terms of cost savings, security, and flexibility.

WHAT WE'VE LEARNED ABOUT MODEL SERVING

The authors of this book have been building ML systems for a variety of enterprise use cases for over a decade. We've worked on projects that are fully outsourced (OpenAI), self-managed on-premise solutions (Tensorflow serving, TorchServe, Nvidia Triton, Ray), fully-managed cloud vendor solutions (AWS Bedrock), and deeply customized vendor solutions (AWS SageMaker).

What we've learned over the years is there is no "one size fits all" or "eternal" architecture. Model algorithms and serving technologies keep evolving, and businesses need to keep pace. Adopting the new technology lets you provide a better user experience and reduce operational cost, helping to keep the business alive and help it thrive.

As a business owner or model serving developer, it's mandatory to have a solid understanding of the fundamentals of model serving, so that you can make sense of new innovations in the industry and assess pros and cons to make the right technical choices.

At the end of day, we don't want you to be locked into any specific frameworks or vendors: we want you to have the ability to understand and adopt new serving technologies quickly, in order to give your business an edge among fierce competition.

Why Optimize Model Serving (Especially for LLMs)?

Once you've successfully deployed your model serving system, the next challenge is cost optimization. This raises key questions: Can we run the model on more cost-effective hardware? Can we improve throughput and reduce latency without upgrading infrastructure? Can we fully utilize the

Cloud vendor’s LLM service? This is where *model serving optimization* comes into play.

Model serving optimization refers to the process for improving the model serving performance, such as reducing serving latency, increasing throughput, and optimizing resource usage. In practice, we do serving optimization to maximize serving efficiency while keeping costs under control.

OPTIMIZATION IS CRITICAL FOR LLMS

LLMs are large and complex enough to demand significant computing power, which can make their operational costs unaffordable for small businesses. For instance, Alphabet chairman John Hennessy told [Reuters in 2023](#) that “running an LLM request can be 10 times more expensive than a traditional keyword search, potentially leading to billions in additional costs.”

So, for LLMs, it’s almost a must to optimize your model serving: doing so lets businesses reduce costs, enhance user experience with faster response times, and gain a competitive edge over companies with slower or more expensive inference solutions.

In the next section, we’ll break down the key concepts of model optimization and walk through some examples to help you get a feel for why an optimized model and serving system—especially for LLMs—can make a huge difference compared to a “raw” model straight from training.

No worries if some of this feels unfamiliar! We’ve got you covered—Chapters 5 through 8 will dive deeper into these concepts with hands-on

examples to make everything clear. So stay tuned!

Why does model serving optimization work?

To get a good grasp of model serving optimization, it helps to first understand the difference between model *training* and model *serving*. A common question people ask is: “Why not just use the same setup for serving as we do for training? After all, both involve running the same model, right?”

Technically, that’s partially true—both processes involve loading the model and executing its architecture. But the goals and requirements are completely different. Let’s break it down in [Table 1-1](#) to see why.

Table 1-1. Fundamental differences between model training and serving

Aspect	Model Training	Model Serving
Stage in ML Lifecycle	Prepares the model	Deploy to production
Objective	Run the model as a process to learn parameters (weights) by minimizing a loss function on the training data.	Run the model to generate prediction (inference) on new input efficiently.
Computation	Extremely compute-intensive, involving iterative model weight updates (including backpropagation and gradient updates)	Focused on efficient forward propagation only (no backpropagation)

Aspect	Model Training	Model Serving
Throughput and Latency	Preferable for high throughput (many samples per second). Large data batches are usually processed in parallel to optimize GPU utilization.	Optimized for low-latency response per request; often operates on single or small batches
Resource Requirement	Requires powerful GPUs/TPUs and distributed training frameworks (such as Fully Sharded Data Parallel (FSDP) and DeepSpeed)	Optimized for low-latency, resource-efficient execution, often on CPUs, edge devices, or inference-specific GPUs (such as NVIDIA TensorRT and ONNX Runtime)

From [Table 1-1](#), you can see why using a model-training framework and system for serving would be inefficient. Instead, adopt model-serving-specific frameworks, such as [NVIDIA Triton Inference Server](#), [vLLM](#) (Virtual LLM), and [SGLang](#).

To give you an example, let’s look at vLLM, an optimized LLM serving engine designed by UC Berkeley’s SkyLab for high-throughput, low-

latency inference. It significantly improves the efficiency of serving LLMs, with 24x higher throughput than basic serving approaches like Hugging Face Transformers or PyTorch's native model execution, which are tailored to model training, static batching, being generic and user friendliness. Online serving, being more specific, poses more opportunities for improvement.

Like other model serving frameworks, vLLM exposes lots of knobs for engineers to tune for optimal model performance. Take a look at this vLLM command for running LLaMA-2 13B on an A100 80GB GPU, with a few optimal configurations:

```
python -m vllm.entrypoints.openai.api_server \
  --model llama-2-13b-chat-hf \
  --dtype bf16 \

  --gpu-memory-utilization 0.9 \
  --max-num-seqs 16
  --max-num-batched-tokens 16384 \
  --tensor-parallel-size 2
```

This vLLM command passes several configurations to vLLM aimed at achieving good throughput and latency on the Nvidia A100 GPU. For example, it sets Paged Attention (which is enabled by default in vLLM) to KV Cache, sets model precision to BF16, sets the max concurrent request

(batch size) to 16 and shares the model across two A100 GPUs. (We'll talk about all of these options in Chapters 6 and 7; it's OK if they are unfamiliar now.)

YOUR SERVING FRAMEWORK'S CONFIGURATION CAN SIGNIFICANTLY IMPACT PERFORMANCE

Every option available in a serving framework represents a potential optimization technique, and the right configurations can make a huge difference in performance.

For example, in our own experiment hosting the DeepSeek R1 model with vLLM, we gained a 15x increase in throughput—jumping from 38 tokens per second to 600 tokens per second—by doing two simple things in vLLM's configuration: enabling FP8 Matrix Learning Accelerator (MLA) kernels and increasing the batch size.

Example: Performance Gain by Optimizing LLM's KV Cache

We've covered a lot of high-level optimization concepts so far, but now it's time for you to see their impact in action.

In this section, we'll share some performance improvements by optimizing the key-value (KV) cache for LLMs using two serving frameworks—vLLM and SGLang. These results will give you a tangible sense of how model optimization translates into real-world improvements.

What is LLM (Transformer) KV (Key-Value) Cache?

Key-value caching is a technique that helps speed up the model inference process by remembering important information (such as [attention](#)) from previous steps. Instead of recomputing everything from scratch, the model reuses what it has already calculated, making text generation much faster and more efficient. [Figure 1-3](#) shows a general KV-cache workflow in the transformer decoding process.

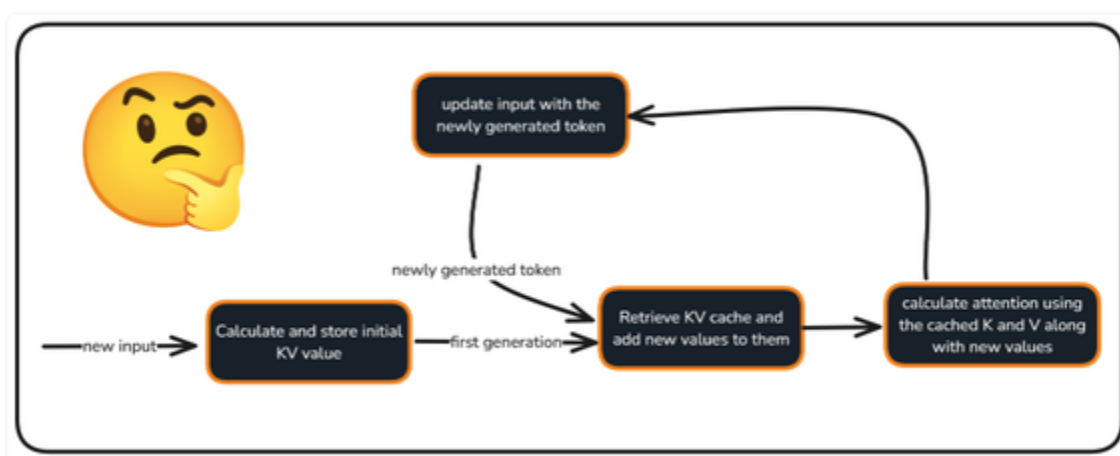


Figure 1-3. How a KV cache works, step by step. (Source: [HuggingFace blog](#))

In [Figure 1-3](#), you can see how the KV cache saves unnecessary attention calculations:

1. When the model sees the input prompt, it calculates and stores it as key-value pairs in the cache.
2. When generating new tokens, instead of starting over to recalculate KV cache from the very beginning, the model retrieves the stored KV cache instead.

3. With the KV cache, the model now can calculate attention efficiently by aligning the cached keys and values with the new query (Q) to compute the new token.
4. The model appends the newly generated token to the existing sequence and repeats the process from step 2 until it is finished generating.

The KV cache makes a big difference to the speed and efficiency of LLM inference, especially for long context and long generations. By saving and reusing past calculations, it avoids the need to start over each time and trades storage space for less computation, making it much faster than the regular way of generating text. Even a vanilla version of a KV cache can lead to a fivefold improvement in LLM serving speed, as the [HuggingFace \(HF\) blog explains](#).

Now that you know what a KV cache is, let's look at what happens when we use vLLM and SGLang to optimize an LLM's KV cache.

vLLM with Paged Attention

Increasing an LLM's throughput requires batching many requests at a time. However, the KV cache memory for each request is huge, and it grows and shrinks dynamically. This means that, if it is managed inefficiently, this memory can be significantly wasted by fragmentation and redundant duplication, limiting the batch size and thus affecting the throughput.

vLLM comes with a good approach, a feature called PagedAttention that manages the KV cache dynamically, like a paging system in an OS, to eliminate fragmentation and enable continuous token generation across multiple requests without memory waste. As a result, research has shown that it improves GPU utilization.

The vLLM team experimented in 2023 by adopting PagedAttention on two settings: LLaMA-7B on an NVIDIA A10G GPU and LLaMA-13B on an NVIDIA A100 GPU (40GB). Their results were impressive, achieving throughput up to 24x higher than HF Transformers and up to 3.5x higher than HF Text Generation Inference (TGI). Figure 1-4 illustrates their findings.

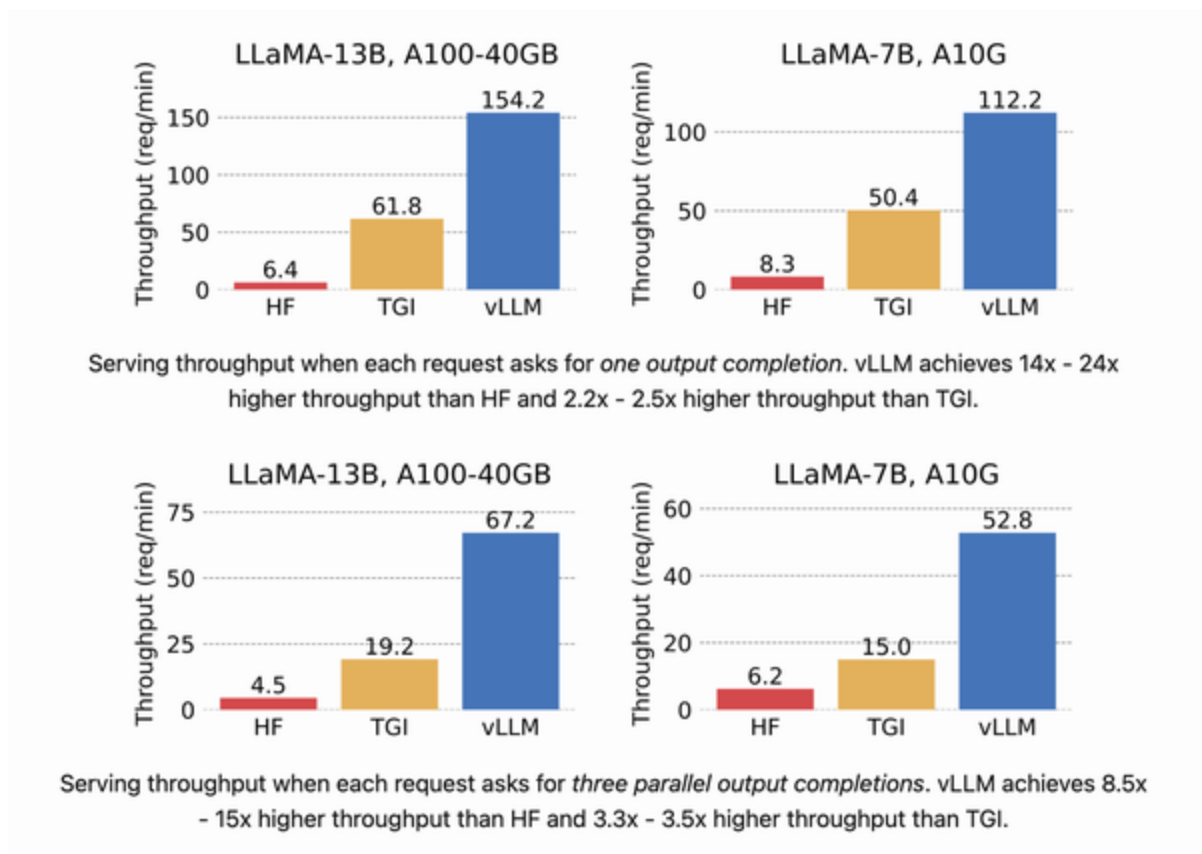


Figure 1-4. vLLM achieves 8.5x - 15x higher throughput than HF and 3.3x - 3.5x higher throughput than TGI. Image source: [vLLM blog](#)

You can use `gpu-memory-utilization`, `max-model-len`, and `block-size` to fine-tune the PagedAttention feature based on model size:

```
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-2-13b-chat-hf \
  --gpu-memory-utilization 0.9 \

  --max-model-len 4096 \
  --block-size 32
```

More details of vLLM configuration can be found in the [vLLM documentation](#).

SGLang with RadixAttention

SGLang is a model-serving framework that focuses on optimizing the KV cache across multiple LLM requests. Since KV cache computation depends on prefix tokens, requests with the same prompt prefix can reuse the KV cache, reducing redundant computation and memory usage.

In existing inference engines, the KV cache for a request is discarded after its own generation is completed and is not reused across multiple calls. If the input prompts of the requests happen to have many common prefixes, recomputing all the KV cache is not efficient and reusing them can vastly reduce the serving latency. So SGLang came up with a feature called RadixAttention, which maintains a LRU cache of the KV cache for all requests within a radix tree; it enables automatic reuse of the KV cache across multiple generation calls efficiently.

As a result, in an experiment on hosting open-weight models (Llama-7B and Mixtral-8x7B models), [researchers found](#) that SGLang improves throughput by up to 6.4x and reduces latency by up to 3.7x.² See more at paper, SGLang: Efficient Execution of Structured Language Model Programs.

I hope these examples give you a clear intuition on how optimizing the KV cache computation can significantly boost model serving performance.

More importantly, I want to emphasize just how significant an impact model optimization can have—especially on LLMs. Given the high cost of GPU-based inference, achieving 6x throughput and 3x lower latency without any additional infrastructure cost is a game-changer.

Model Serving Paradigms

In this section, we'll explore the most common model serving paradigms and their real-world applications. We'll introduce these designs progressively—starting with simple on-device and single-model serving, then expanding to more complex multimodel architectures, each building upon the previous concept.

Along the way, we'll share insights from our real-world experience to help you understand the trade-offs of each approach. By the end of this section, you'll have a clear sense of which serving solution best fits your business needs and a foundational understanding of high-level design paradigms for structuring it. The practical design principles, along with hands-on examples of model serving systems, will be covered in Chapters 2, 3, and 4.

On-Device (Edge) Serving

On-device serving refers to running the model directly on user-side devices, rather than relying on remote web services. This setup allows applications to process data locally—where it is generated—on devices such as smartphones, drones, robots, cameras, and VR headsets, enabling real-time computation without the need for constant internet connectivity. [Figure 1-5](#) highlights the four key benefits of on-device AI.



Figure 1-5. The benefits of on-device model serving (Image source: [DeepLearning.AI](#)).

Given these benefits, on-device model serving has been widely adopted across various applications. Examples include smartphone face ID, AI-powered background-noise reduction in calls, intelligent robotic vacuum cleaners, and drones equipped with AI-assisted flight mode, all leveraging local AI processing for faster, more efficient performance.

On-Device Serving Design

Now let's explore how on-device model serving operates. We'll begin by examining the key components of an on-device AI application, followed by an explanation of the deployment process required to prepare a model for different devices. The core concepts are illustrated in [Figure 1-6](#).

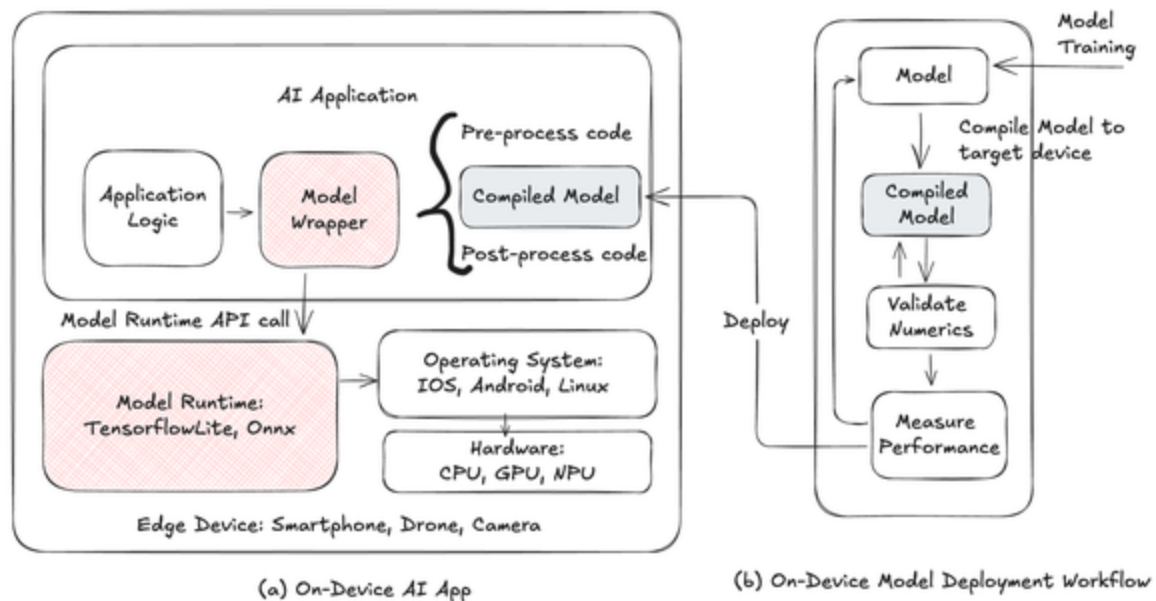


Figure 1-6. On-device AI app design and model deployment workflow.

In the high-level on-device AI app design shown in [Figure 1-6](#) (a), model runtime and model wrapper are the key components.

A *model runtime* is a specialized software framework designed to execute ML models efficiently on different hardware platforms. It serves as the intermediary layer between the trained model and the device's hardware, optimizing the inference process to maximize speed, efficiency, and resource utilization.

The model runtime plays a crucial role in on-device AI by abstracting the complexities of model execution across different hardware (like smartphones, drones, and robots) and operating systems (iOS, Android, Linux). It also provides hardware-specific optimizations and acceleration techniques to improve inference performance.

For example, the model runtime can leverage specialized hardware (like CPUs, GPUs, NPUs, and TPUs) for efficient inference. It also supports *delegates* (such as GPU Delegate, NNAPI for Android, and Core ML for iOS) to optimize performance.

A well-designed model runtime allows developers to focus on building application features rather than worrying about hardware compatibility or model execution details. As of this writing, some of the most popular model runtimes include TensorFlow Lite (TFLite), ONNX Runtime (ORT) and Core ML.

As AI app developers, we use model runtimes to execute models efficiently and encapsulate model-specific execution logic into a reusable module often called a *model wrapper*. This wrapper abstracts low-level details, making it easier to integrate, update, and optimize models within applications.

A model wrapper is a component implemented by the application developers to provide the model inference interfaces (functions) for the

application logic to execute the model. The model wrapper encapsulates the details of how to interact with the model runtime, such as preprocessing the input data to model input, load the model into the model runtime, execute the model from the model runtime, and postprocess the output.

In the workflow shown in [Figure 1-6](#) (a), the application logic first asks the model wrapper to execute a model with given data. The model wrapper takes care of data conversion and calls the model runtime to execute the model. The model runtime then handles the execution using the local hardware.

Preparing a model for on-device serving

[Figure 1-6](#) (b) describes the high-level workflow of getting a model ready, from the training process to operating on-device.

Once the model has been trained, you first need to convert it from its training format into the format the model runtime requires. For example, if you're using the Tensorflow Lite runtime, you can convert (compile) a Resnet pytorch model into Tensorflow Lite's file format (*.tflite*) by using the following command:

```
ai_edge_torch.convert(resnet18.eval(), sample_in)
```


Once the model is compiled, the next step is to validate its numerical accuracy on the target device. This ensures that converting the model and optimizing it for on-device execution haven't degraded its accuracy. The most common validation method is to run the same model inputs on both the training server and the local device, then compare the output differences.

Next, you'll need to measure the model's performance to ensure that the local hardware can execute the model efficiently while meeting the business requirements. Once you've validated that, you can package the optimized model as part of the application installation or update for release.

This deployment process may seem tedious, but tools like [Qualcomm AI Hub](#) help automate all the key steps, including on-device model compilation, optimization, and validation, making the workflow much more efficient.

Benefits, Limitations, and Challenges

Running models directly on devices offers benefits like low latency, offline capabilities, and better privacy, but it comes with several limitations and challenges and often requires cloud-based model serving. Here's a breakdown of the key challenges, along with examples.

Computation and Storage Constraints

Smartphones, IoT hardware, and embedded systems have limited CPU, GPU, and memory resources, making it difficult to run large, complex models. For example, running a large transformer-based language model like GPT on a smartphone is infeasible due to memory, storage, and processing-power constraints.

Power Consumption

Running AI models, especially deep neural networks, can be power-intensive, draining battery life quickly. For example, we don't run real-time video enhancement (from low to high resolution) locally, since it consumes excessive amounts of power.

Limited Update and Maintenance Capabilities

On-device models are hard to update frequently. When improvements are made to the model, each device must receive and install an update. For example, a banking app using an on-device fraud-detection model would need to release frequent app updates to improve its model's accuracy.

Inconsistent Hardware Support

Different devices have different hardware capabilities: some support NPUs, others only CPUs. This can make it hard to deploy optimized models across all devices. For example, a model optimized for

Apple's Neural Engine on an iPhone might not run efficiently on an Android device with a Qualcomm Snapdragon processor.

When faced with these challenges, we often choose to either move the entire model serving to the cloud or adopt a hybrid approach, where lightweight AI processing happens on-device and more complex computations are offloaded to the cloud. Next, let's take a closer look at cloud-based model serving approaches.

Single Model Service

In this section, I'll introduce you to the classic Single-Model Service design pattern, which is also the most widely used cloud-based model-serving approach.

In this design (diagrammed in [Figure 1-7](#)), each model, model version, or model type is deployed as a dedicated web service, exposing a prediction API over HTTP or gRPC for server-side execution. This setup ensures scalability, isolation, and flexibility in model deployment.

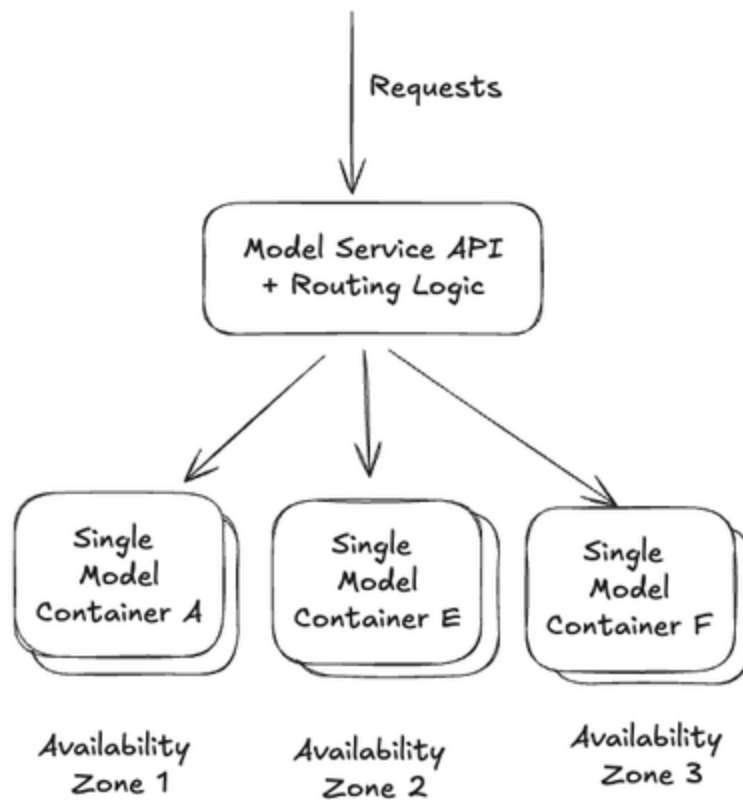


Figure 1-7. Single-model service architecture.

From [Figure 1-7](#), you can see that the Single-Model Service architecture follows a [standard microservices design](#). It has an API component in front to receive web prediction requests and route them to one of its backend workers/containers. The backend workers and containers perform the actual model execution and return the result.

CONTAINERIZATION IS THE FOUNDATION OF MODERN MODEL SERVING

Containerization means creating an isolated environment (or *container*) that packages an application along with all its dependencies, libraries, and configurations. Using a container ensures that the application runs consistently across different computing environments, whether on a developer's laptop, a test server, or in production.

In modern model serving, most server-side model execution happens inside a container. It's a common practice to encapsulate the model management, resource management and model execution logic into a [Docker container](#) (or [Kubernetes Pod](#)), and expose the model's prediction/serving function via a gRPC or HTTP interface.

Docker is the most popular containerization technology as we write this book, so we use Docker containers throughout the book, including code examples.

Now let's look at [Figure 1-8](#) to see what's inside a single model serving container.

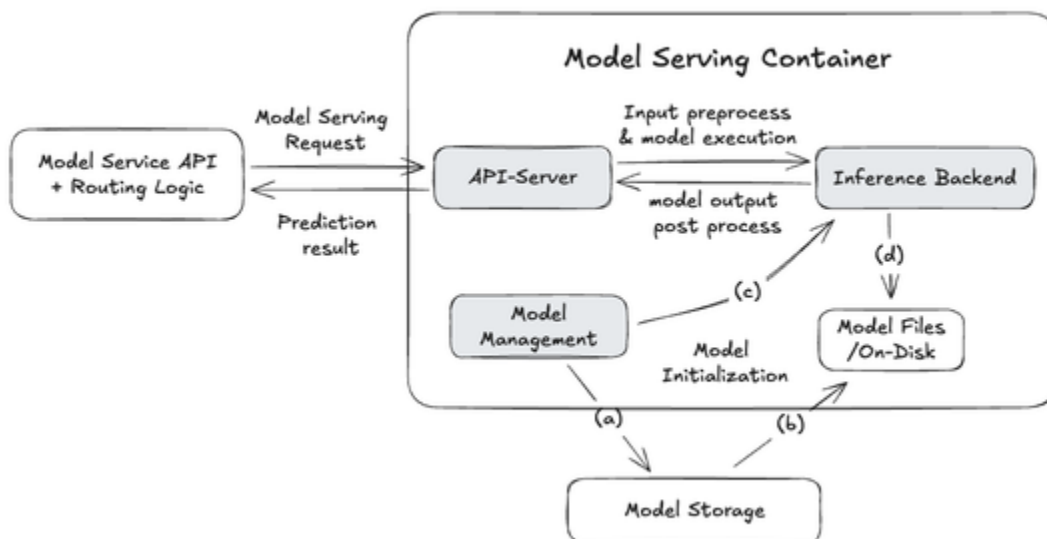


Figure 1-8. Single-model serving container design

You can see that there are three main components in the model-serving container ([Figure 1-8](#)): API-Server, Model Management, and Inference Backend. Inference Backend is the model execution component. It usually leverages a model-serving framework (like TensorFlow Serving or TorchServe) or libraries (like vLLM or TensorRT-LLM) to run model serving. Model Management is the component responsible for preparing the model for serving. This includes downloading the model, extracting it to local storage, and loading it into the inference backend (initialization steps a through d in [Figure 1-8](#)). API-Server exposes the model inference functionality over HTTP or gRPC, enabling external applications to send prediction requests.

For model deployment, we expect the model-training pipeline or the Ops (Operations) team to upload the trained model to cloud storage. The Model Management component then automatically refreshes the model in each serving container, ensuring that the latest version is deployed seamlessly.

Routing Choices in Single Model Serving

One of the key responsibilities of the model service API component (in [Figure 1-7](#)) is to ensure all the model serving containers are saturated with the prediction workloads *equally*. It sounds like a problem that can be solved easily using a “round-robin” load-balancing strategy. For example, if you have three containers (c1, c2, and c3), the first prediction request would go to c1, the second to c2, the third to c3, and then it starts over again.

However, round-robin routing has a key limitation—it doesn’t account for varying request processing times. In many cases, prediction requests have different computational costs based on input size and complexity. For example, an LLM inference request with a large text payload takes longer to process than one with a shorter input. An image model processing high-resolution images consumes more GPU/CPU resources than one processing smaller images.

Round-robin load balancing creates inefficiencies because of this variation: for instance, you might have a situation where container c1 is still processing a large request and container c2 has already finished its work, but the next request is still routed to c1, creating a bottleneck.

To address this issue, we need more sophisticated routing strategies that take into account processing time, resource availability, and request complexity. Here are some frequently used routing choices:

Weighted Round Robin

Servers (bigger) with higher weights receive more requests

Least Connections

Directs requests to the server with the fewest active connections

Least Response Time

Routes requests to the server with the lowest response time

Dynamic Load Balancing

Uses metrics like CPU, GPU, memory usage, or queue length to distribute requests

Horizontal and Vertical Scaling

As prediction-request traffic increases, a single serving container may struggle to process requests quickly. So we need to deploy more instances of the serving containers (we'll refer to them as just *instances*) across multiple machines to handle the growing number of concurrent users with minimal delays. This is called *horizontal scaling*, or *scaling out*.

In a production serving environment, we enable *autoscaling*, so that the infrastructure management system automatically provisions more instances based on server resource utilization. The system monitors key metrics such as CPU and memory usage, inference latency, and number of active requests.

When monitored metrics exceed their predefined thresholds, the infrastructure management system spins up additional instances dynamically. For example, if you use [Kubernetes](#) to manage your compute cluster, its [Horizontal Pod Autoscaling](#) (HPA) can automatically adjust the number of serving pods based on real-time demand.

For large deep-learning models, such as GPT-4 and Llama 2-70B, a single machine may not have enough GPU memory to load the model. In these cases, we use *vertical scaling* (*scaling up*). This means deploying the model on more powerful GPUs (such as NVIDIA H100 or A100) and using distributed serving across multiple GPUs or multiple machines to split the workload.

Distributed model serving can be complex, but modern frameworks like vLLM simplify the process by abstracting away the low-level details. You can easily specify the number of GPUs to use when launching a model. For example, to serve a Llama model on 4 GPUs using vLLM, you can run this command:

```
python -m vllm.entrypoints.api_server \
    --model meta-llama/Llama-2-13b-hf \
    --tensor-parallel-size 4 \
```

PRIORITIZING INTRA-NODE SERVING (MULTIPLE GPUS ON ONE MACHINE) OVER INTER-NODE SERVING (MULTIPLE MACHINES)

For better serving performance, easier management, and lower latency, it's generally preferable to run large models on a single machine with multiple GPUs (*intra-node serving*) rather than distributing them across multiple machines (*inter-node serving*).

Inter-node serving introduces network overhead, increasing latency and complexity in synchronization. Whenever possible, we recommend you optimize model execution to fit within a single machine, to maximize efficiency.

We'll dive deeper into these model optimization techniques in Chapters 6 and 7.

Benefits, Limitations, and Challenges

The Single Model Service approach is our favorite because of its simplicity and versatility. It works seamlessly with any type of model, small or large, from LLMs to deep learning models, making it a reliable and scalable choice for model serving.

From a production and operation perspective, we use Single Model Service whenever possible for the following reasons:

- It offers the best performance, with no resource contention and lower latency.
- Scaling is easy, since each model can scale independently.
- Deployment and debugging are simpler thanks to isolated logs, metrics, and updates.
- It's more reliable since if one model crashes, it doesn't affect others.
- Optimized hardware per model allows for better cost control.

Although the Single Model Serving approach is great, it falls short, particularly in resource efficiency and cost.

Imagine you're running an agent platform where customers can create their own agents and deploy custom models. If you have 100 customers and each deploys 10 models, that means your platform would be hosting 1,000 separate model services. This setup isn't scalable—the overhead of maintaining, patching, deploying, and monitoring so many services would be overwhelming. Worse, many uploaded models might never be used, leading to wasted compute resources and unnecessary infrastructure costs.

To reduce operational complexity and maximize resource utilization, you need a more efficient solution—one that allows multiple models to share compute resources dynamically, instead of running a separate service for each.

This brings us to Multi-Model Service, a design approach that enables cost-effective, scalable model serving.

Multi-Model Service

Multi-Model Service is designed to co-host multiple models in one serving container, sharing GPU/CPU and memory across multiple models, and to load and unload models dynamically based on the incoming traffic. This can help you save significantly on costs and achieve the best price performance.

Multi-Model Service is an ideal solution when you need to serve a large number of models efficiently without loading them all into memory at once. Instead of dedicating a separate container to each model, this approach has models *share* compute resources within a single serving instance.

Going back to our agent-platform use case, instead of loading all 1,000 models into GPU and memory simultaneously, with Multi-Model Service, you could adopt an on-demand approach: Load the model only when a customer requests a prediction, and unload it when it's inactive or when memory needs to be freed up for a new model. This significantly reduces infrastructure costs and improves resource utilization, making the model serving far more scalable.

You might wonder how it's possible to host multiple models in the same serving container when they may have been trained using different frameworks and require different execution parameters?

The answer lies in the design of the Model Serving container, which manages model loading, execution, and unloading dynamically. Let's explore its architecture, shown in [Figure 1-9](#).

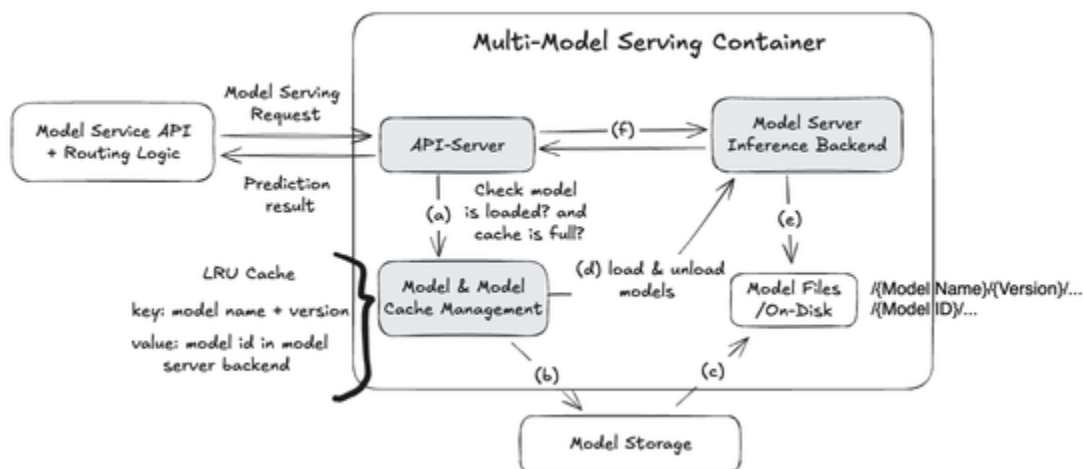


Figure 1-9. Multi-Model Serving Container Design

If you compare [Figure 1-8](#) and [Figure 1-9](#)—the Single Model and Multi-Model serving containers, you can see that the Multi-Model Serving Container has *two* components: a Model Server Inference Backend and Model Cache Management.

The Model Server Inference Backend is designed to handle multiple types of models in a black-box manner. Regardless of the model algorithm and

version, it can serve these models with a unified web prediction API. The trick here is that the model server contains multiple types of inference backends internally, and each backend handles one type of model (for example, TensorFlow, ONNX, or PyTorch).

Fortunately, you don't have to build a model server from scratch—there are robust open-source solutions available, such as NVIDIA Triton Inference Server. In practice, we deploy systems like Triton inside model serving containers (or Kubernetes pods), using it as the Model Server Inference Backend to efficiently host and manage multiple models in GPU and memory simultaneously.

NOTE

Triton Inference Server is one of the most popular open-source model serving solutions, and it's also one of our go-to tools, especially for multi-model use cases.

Triton can manage and serve any number or mix of models, constrained only by system disk and memory resources. It provides a unified API on top of various model formats, including: TensorRT, TensorFlow GraphDef, TensorFlow SavedModel, ONNX, PyTorch, Caffe2 NetDef and more. See more about Triton from [Triton Architecture Blog](#).

The Model Cache Management component has two responsibilities. First, it loads a given model from model storage into the Model Server Inference Backend. Second, it prevents the container resources from model overload by maintaining a Least Recently Used (LRU) cache to track all the modeled

models, so it can unload the least-used models when the container's resource utilization is high.

Now, let's look at an end-to-end on-demand model serving workflow. When a prediction request on model A hits the container, model management will do three things for the request:

- If model A is loaded already, it forwards the request to the model server backend to execute the model and return the result.
- If model A is not loaded, the management component will download the model from model storage, load it into the model server, and forward the request.
- If memory and disk consumption is above a specified threshold (for example, 80%), the management component will find the least used model in its cache, unload it from the server backend, and remove the model from cache and disk.

Routing and autoscaling in Multi-Model Service

Sharing resources to host multiple models is efficient and cost-effective, but it also introduces challenges in routing prediction requests to the right models and scaling them effectively. Let's dive into these challenges and explore potential solutions.

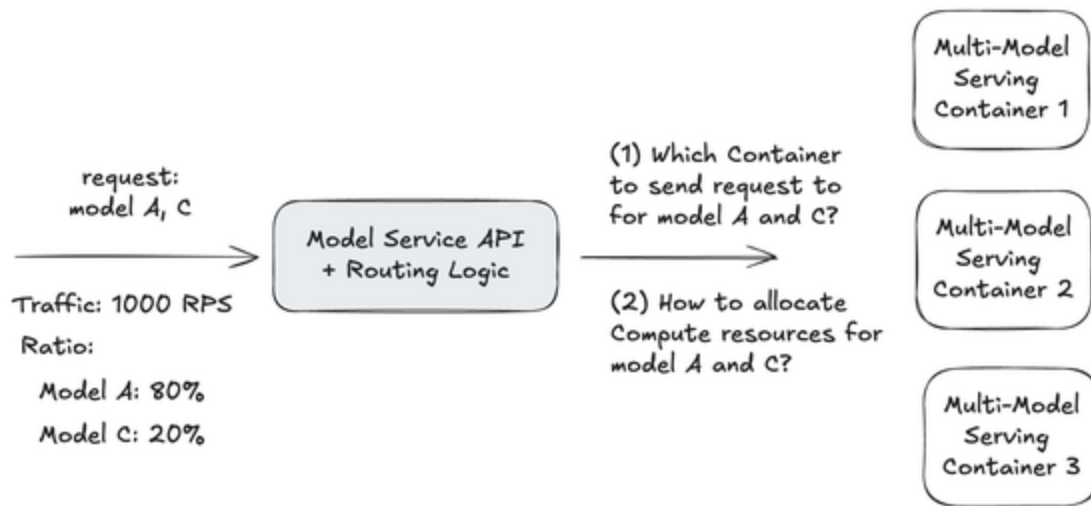


Figure 1-10. Serving and scaling challenges in multi-model serving

We can see two problems in [Figure 1-10](#).

The first problem is about routing. Since models can be hosted in any of the serving containers, we want to route the prediction request to the container which already loaded the model. Otherwise, we add extra latency to the request issue: “cold start” (pending the request for model loading) and “model swapping” (removing old models out of memory for the new requested model).

The second problem has to do with per-model scaling. Not all models will take the same amount of traffic; some models will be used more frequently than others. We want to have more model instances in our multi-model containers for these “hot” models.

To solve the routing and model-scaling issues, we need to enhance the routing component at the frontend API layer (see [Figure 1-11](#)).

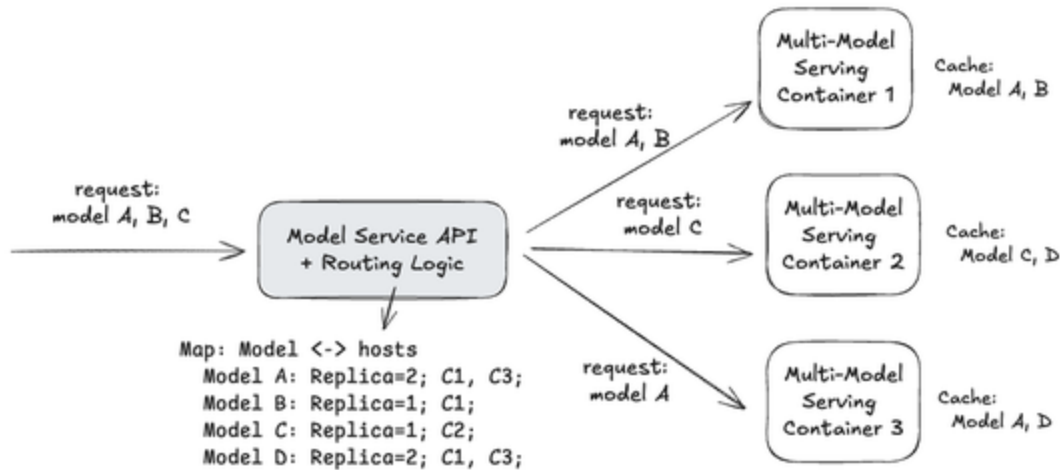


Figure 1-11. Route model server requests with host/model map and model replica counts

In [Figure 1-11](#), we add two features to address the model-routing and autoscaling problems that arise in multi-model serving. First, we add an attribute “replica” to the model’s metadata. The number of replicas defines how many instances of this model we want to host in the backend containers. Second, we introduce a route map in the routing layer, so it knows to which containers it should route the request.

For example, as [Figure 1-11](#) shows, for a given Model A with a current replica count of 2, the routing logic will choose two containers (for example, c1 and c3), to host it. Then it will route all future model A requests to c1 and c3. Depending on Model A’s traffic ratio, the routing component can update its replica count to autoscale Model A up and down.

Benefits, Limitations, and Challenges

As we've explained, multi-model service is the most cost-efficient serving method since models can share the same compute resources, which is extremely powerful when you don't have to host all the models at the same time.

However, multi-model service tends to struggle in the following situations:

- When the models are too large to fit in one GPU. Either there is no room to share or lots of model unloading is required to make memory space available. This causes a lot of model loading overhead and cold-start latency for future models.
- When individual models have high traffic levels, which requires low-latency, always-loaded models. Single-model service is better in these cases.
- When the models have different security policies.
- When operational complexity is high. For instance, deployment, debugging, and monitoring are complicated due to the number of moving pieces in the system, such as model cache management, per-model routing and scaling, model compatibility, and dependency conflicts.

At this point, you might already have noticed that the multi-model and single-model serving approaches are complementary. In practice, we compose different serving approaches together to make up a platform, so

we can address different serving use cases under the same roof. The next section discusses model serving platforms.

Model Serving Platforms

As a business grows, its model-serving demand surges, and its requirements become increasingly complex. Simply putting single-model and multi-model services together is no longer good enough. These serving systems face two major challenges.

First, many tasks now require multiple models to work together. For instance, a voice assistant like Apple Siri must integrate speech recognition, natural language processing (NLP), recommendation models, and text-to-speech synthesis to respond to a single voice command. Managing these multi-model interactions efficiently is crucial for seamless performance.

Second, when a growing number of AI applications are running on one platform, optimizing compute resources becomes a critical challenge. You need an efficient strategy to allocate GPUs, CPUs, and memory across models while ensuring scalability, cost-effectiveness, and minimal resource contention.

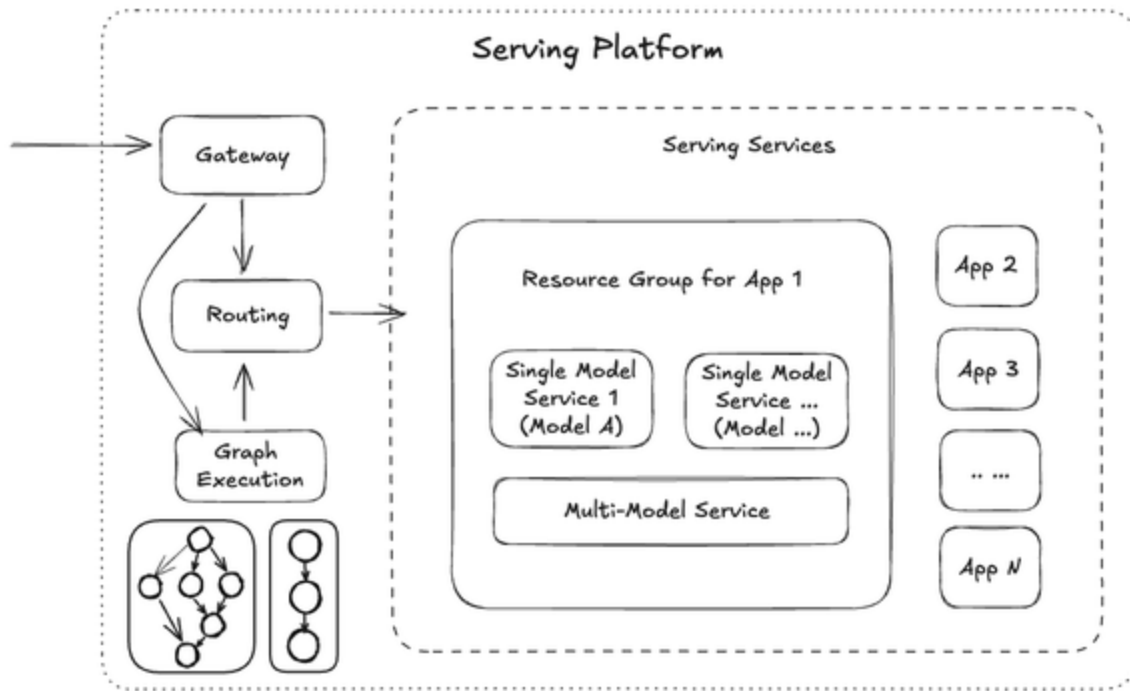


Figure 1-12. Model Serving Platform Design

The model serving platform design in [Figure 1-12](#) gives you a sense of how a model serving platform addresses the aforementioned challenges. First, it uses resource groups to segregate the prediction workload for different applications. For example, we see two single model services and one multi-model service deployed in the App1 resource group. Each resource group is allocated CPU, GPU, and memory quotas based on business requirements to optimize performance and cost.

Second, the design has a graph execution component (for example, [Airflow](#) or [Ray](#)) to support multistep inference workflows. Each app team will define their own inference workflow and deploy it to the graph execution engine. Then the graph engine runs the workflow. For each inference step in

the workflow, the graph engine calls the routing component to find the right serving group and service to run inference.

[Figure 1-12](#) provides a high-level overview for a model-serving platform. In practice, this design could be a lot more complicated. For example, a serving platform must have security and access-control components, metric and monitoring components, deployment and DevOps integration, and more. Since these components are not core to model serving, we've omitted them from this diagram to keep the focus on the main serving concepts.

Summary

In this chapter, we explored the structure of ML from an engineering perspective, viewing them as executable components rather than just static data files. A model consists of three key elements: model data (weights, biases, and configurations), model architecture (the structure of layers and operations of the model), and model execution code (which loads and runs the model for inference).

Next, we introduced model serving, which involves deploying models in production environments to process real-time data and generate predictions. We highlighted three deployment scenarios—on-device, on-premises, and cloud-based model serving—and outlined the key considerations for model

serving, such as scalability, latency, monitoring, security, and cost optimization.

Then we explored why understanding and optimizing model serving is essential, even when leveraging cloud vendor solutions or large language models (LLMs). While cloud-based model serving is convenient, businesses must still navigate complex integrations, cost trade-offs, and security concerns. Similarly, LLMs are powerful but often require multi-model pipelines, fine-tuning capabilities, and cost-efficient deployment strategies to be practical in real-world applications.

We also introduced model serving optimization, particularly for LLMs, where efficiency gains can lead to significant cost reductions and performance improvements. By optimizing techniques such as KV Cache with Paged Attention (vLLM) and Radix Attention (SGLang), businesses can achieve six times higher throughput and a third of the latency without incurring additional infrastructure costs. Since LLM inference is computationally expensive, serving optimization is not just an improvement—it's a necessity for cost-effective AI deployment.

Finally, we explored the most common model-serving paradigms, from on-device (edge) serving to single-model service, multi-model, and full-model serving platforms. Each paradigm has its strengths and trade-offs, depending on factors such as latency, cost, scalability, and complexity.

We hope this chapter has convinced you that model serving is critical—and that it’s an engineering-driven discipline, focusing on efficiently deploying and integrating models into real-world applications, rather than on the intricacies of training models or designing algorithms.

In the next two chapters, we’ll dive into hands-on examples to see how model serving is built and operates in practice. We’ll start with LLM model serving in Chapter 2, then expand to a broader model-serving system design and implementation in Chapter 3.

- We use the terms *model serving*, *model inference*, and *prediction* interchangeably throughout this book.
- ! The throughput and latency improvements SGLang made come not only from KV cache reuse, but also from exploiting parallelism within a single program and from faster constrained decoding.

Chapter 2. Challenges When Serving LLMs

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at sgrey@oreilly.com.

So far in this book, we have demonstrated the core concepts of model serving, several architectural patterns for serving ML models, and the analysis of the trade-offs involved in deploying models at scale. By now, we hope you have a strong understanding of model-serving paradigms, because we are about to take a significant step into a different realm. In this chapter, we will shift our focus to one of the fastest-growing fields in the AI world: optimizing large language models (LLMs) for serving.

Since the rise of ChatGPT in late 2022, LLMs have transformed how AI is applied in real-world scenarios, from chatbots and code generation to advanced reasoning and decision-making systems. However, their sheer size, computational demands, and unique serving requirements introduce challenges that can go far beyond classic model-serving techniques. From novel ideas to widely adopted frameworks, the field of optimizing LLMs for faster and more efficient serving performance has evolved at an unprecedented pace. It can be daunting: anyone not familiar with this area can easily get overwhelmed. For example:

- When reading technical blogs, you may wonder: “What is this vLLM framework that has gained popularity in just a year or two and has already been adopted in so many places?”
- When reading research papers, you may ask: “How does FlashAttention work, and how can I optimize it at the hardware level to speed up LLM inference?”
- When following AI news, you may come across a new term called MLA and ask: “How much more efficiently does the DeepSeek V3 model run than other attention mechanisms in the serving phase?”

In the following chapters, we will introduce you to all these advancements. But before we go into each technique, we need to establish a strong foundational understanding and gradually move you from basic concepts to more complex techniques. This chapter bridges between all the serving concepts, principles, and paradigms you’ve seen in the prior chapters and

all the advanced LLM optimization techniques we will cover in the following chapters.

You first need to understand the landscape of LLM serving—why it is important, what the hardware requirements are, and how to build your intuition about model optimization. Specifically, we will cover why serving an LLM efficiently can be crucial to the success of your application and business at the very beginning. We'll look at modern hardware's role in LLM serving next, exploring AI accelerators such as GPUs to understand their intricacies of memory, compute capability, and interconnect functionality. Then we'll cover:

- the major bottlenecks in LLM serving and how to mitigate them
- constraints when loading LLMs for serving
- bottlenecks when executing LLMs, specifically in Prefill and Decode
- why model serving can become bottlenecked in different phases

To discuss the last point, we introduce a new concept called *arithmetic intensity*.

Understanding these concepts is essential because, without the fundamentals, optimizing LLM serving can become a tiring trial-and-error experiment. It's easy to get stuck in local optima without knowing it, or just knowing how but not why.

By the end of this chapter, you will have the intuition you need to navigate the world of LLM optimization with confidence. The next chapters will build upon this foundation by going through the various techniques needed to improve performance, reduce costs, and deploy LLMs efficiently for the workload your application requires in production in greater detail.

Why Optimizing LLM Serving is Important

The previous chapters have covered how to make ML models functionally operational and well-designed when deploying them to production. Another crucial aspect is ensuring that the ML model runs quickly and efficiently in a production environment. This is especially important for serving LLMs, which require substantial compute power and memory from hardware.

To better understand their impact, we categorize the key factors into three aspects:

- Customer experience
- Cost efficiency
- Scalability, peak load handling, and feasibility

Customer experience

Customer experience is key to any product's success and can be highly correlated with model response latency. Imagine asking a question in ChatGPT and having to wait over 20 seconds to receive the first token. Such a delay is unsatisfactory, and customers can easily become frustrated while waiting. If we can optimize the serving performance to reduce the model's latency from 20 seconds to just 1 second, while keeping the same hardware setup and overall throughput, that would be a game-changer for the product's success.

However, extremely fast response times may not always yield significant benefits. For example, if we reduce the time to receive the first token from 0.1 seconds to 0.01 seconds, the difference is so small that it doesn't register in our human perception. In this case, if we can trade off some latency for higher throughput, increasing it from 0.01 seconds to 0.1 seconds, we will achieve better cost efficiency.

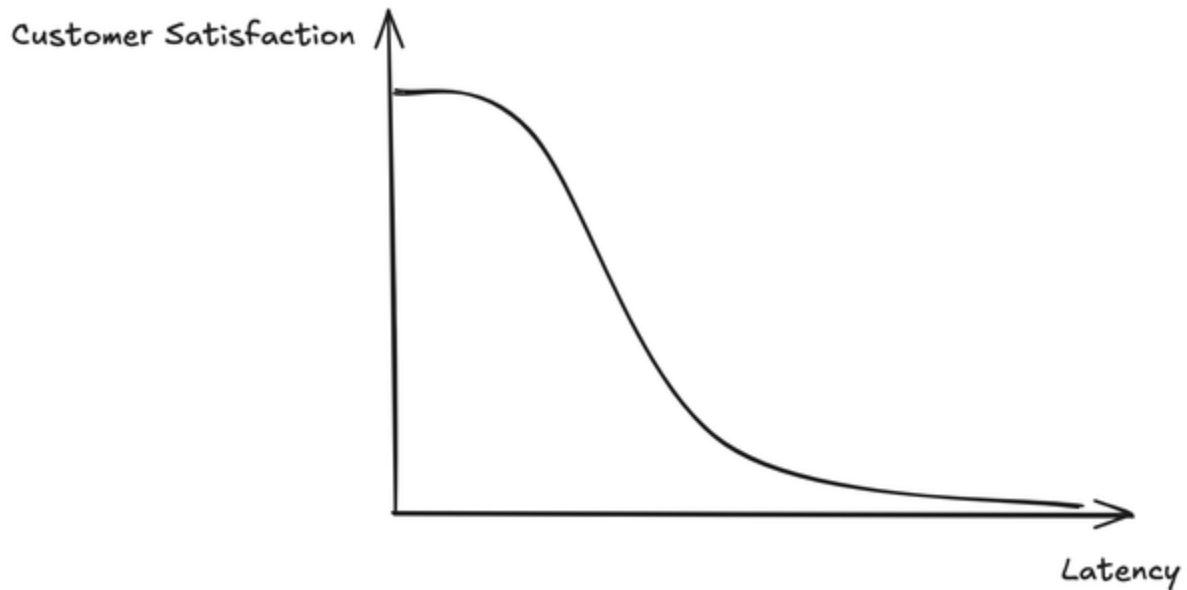


Figure 2-1. Model response latency vs. customer satisfaction

Figure 2-1 illustrates the relationship between latency vs. customer satisfaction. As you can see, the two are inversely correlated—when latency is high, customer satisfaction drops significantly, approaching zero. As latency decreases (moving from right to left along the X axis), customer satisfaction improves. However, as latency continues to decrease, the incremental gains in customer satisfaction become a lot less pronounced.

Another aspect of customer experience is generation quality. Within the same family or similar families of models, the larger models generally produce higher-quality responses. For example, without optimization, we might be limited to an 8-billion-parameter model to meet latency requirements. However, after extensively optimizing the whole model-serving system, we could achieve the same latency and throughput on the

same hardware setup while leveraging a model with 32 billion or even 70 billion parameters. This can significantly improve response quality.

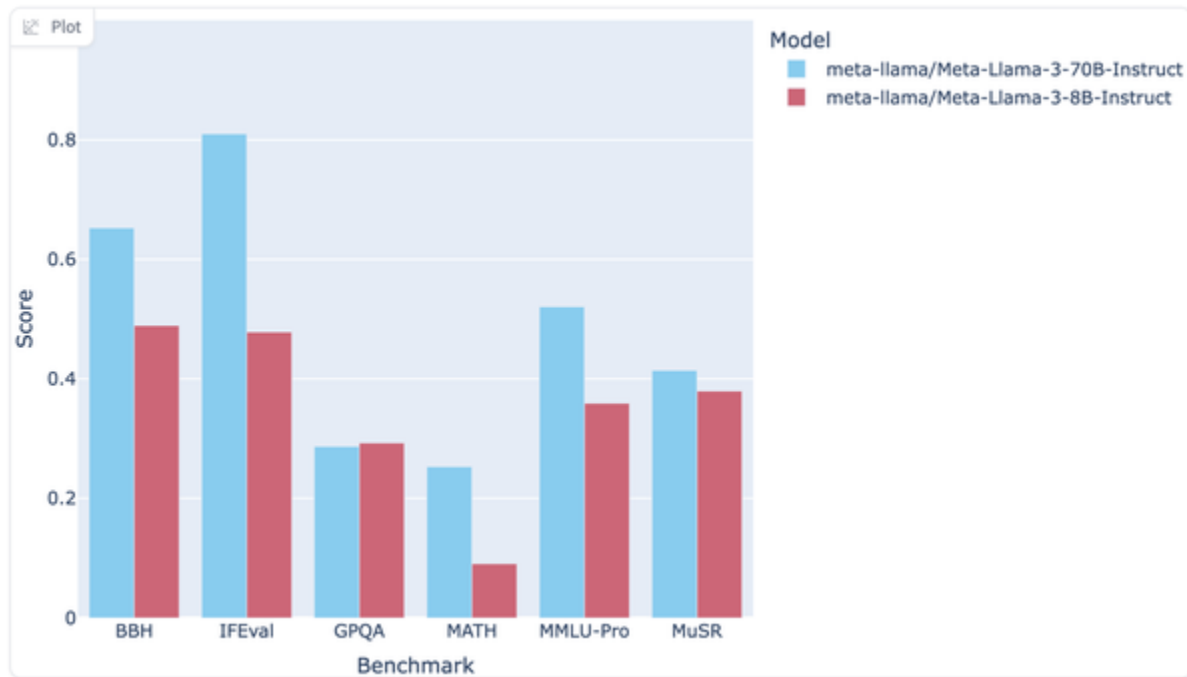


Figure 2-2. Model quality score comparison between Llama-3-70B and Llama-3-8B. (Source: [Open LLM Leaderboard Model Comparator](#))

[Figure 2-2](#) compares the benchmark scores of Llama-3 8B and 70B, demonstrating how larger models outperform smaller models in the same family.

Cost efficiency

Beyond delivering a great customer experience, a product must also be financially viable. No matter how powerful or valuable an AI system is, it cannot succeed if it is too costly to operate at scale. Among all the costs of

AI development and operation, model inference is the most expensive component. This might seem surprising, as training often appears to be the dominant cost—especially with reports stating that training costs for LLMs like GPT-4 exceeded \$50 million.

However, as shown in [Figure 2-3](#), in which *Signal Integrity Journal* [projects AI-chip revenue trends](#) from 2024 to 2034, inference hardware consumption already surpasses training today—and the gap is expected to widen even further. This reflects the growing demand for efficient inference infrastructure, as AI applications scale to serve millions of users in real time.

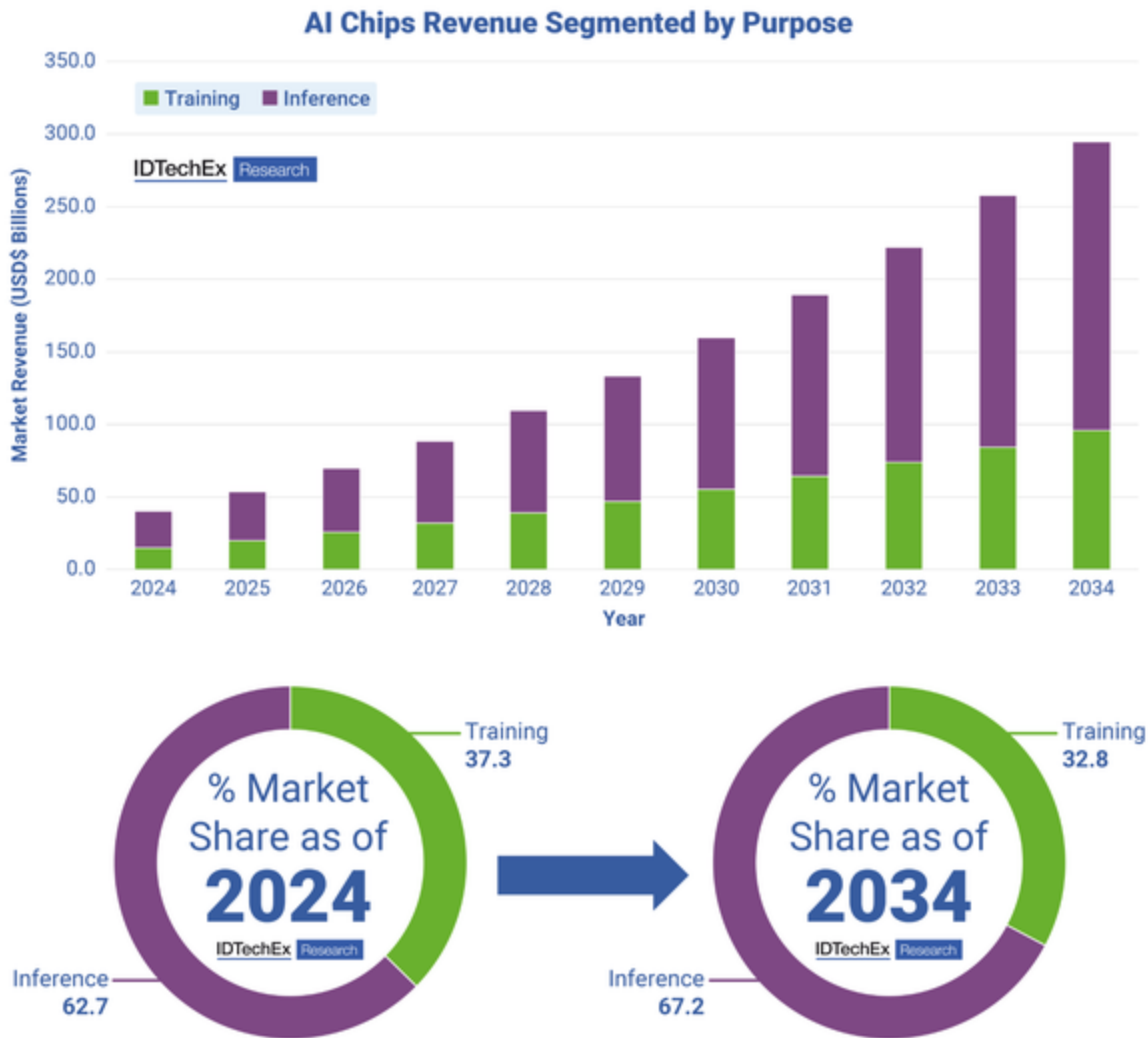


Figure 2-3. AI-chip revenue projected growth and composition change between training and inference. (Source: [Signal Integrity Journal](#))

To better understand the dominance of model serving costs in [Figure 2-3](#), let's do some further analysis. Training costs are primarily upfront investments, though some ongoing retraining and fine-tuning costs remain, whereas inference costs scale continuously as usage picks up. Once a model is deployed to production, every single query or interaction incurs ongoing expenses, which accumulate fast as adoption continues to grow.

Also, at the beginning, a few major players dominated foundation-model training. Recently, however, a growing number of AI labs and open-source efforts are also making significant contributions and advancements—though these still account for a small percentage of all the companies using LLMs for business. The vast majority either fine-tune existing models or enhance them with techniques like Retrieval-Augmented Generation (RAG), which can significantly reduce the cost compared to training a foundation model from scratch.

Meanwhile, the demand for inference is nonstop. As GenAI applications proliferate and are applied to more and more industries, the computational burden of LLMs in production is skyrocketing. The rise of AI agents and complex workflows further amplifies this trend, as these systems often require multiple LLMs and embedding model calls within a single workflow. This compounding effect drives inference costs even higher, making efficient model serving a critical challenge for the future of AI.

By optimizing LLM serving, even using the same hardware and achieving similar latency, businesses can potentially achieve more throughput, serving more customers at the same time. This reduces hardware usage when they horizontally scale the serving system, saving a tremendous amount of money! Another example: an optimized model might be able to run on a lower-grade chip instead of a more advanced and expensive chip, while keeping similar latency and throughput. This also saves money.

Scalability, peak load handling, and feasibility

When a model is deployed in production, GPU demand scales with customer growth. An optimized inference solution enhances customer experience and reduces GPU costs, as you've seen. It also determines whether the system can scale efficiently under heavy traffic, especially when GPU availability is limited—and the industry is indeed just experiencing that.

For example, let's say an LLM-powered sales agent runs smoothly, with stable traffic, most of the year. However, on Black Friday, demand can surge by 400% or more. A less optimized system may really struggle to scale to handle the sudden increase in traffic, leading to bottlenecks, degraded latency, and even request failures. The ability to handle peak traffic reliably is crucial for a resilient and scalable production system. This is a real-world problem we have personally experienced before, and getting the setup right can require a lot of tuning and performance testing to ensure service availability in those periods.

As mentioned in the last section, optimized models can be deployed on lower-grade chips, providing greater flexibility. Even with access to major cloud providers like AWS, Google Cloud, and Azure nowadays, high-end GPUs are not always readily available in every region. The ability to run models efficiently across a broader range of hardware, rather than being constrained to specific high-end GPUs, can be a key enabler for businesses

expanding into new markets. We have frequently encountered this issue when deploying applications around the world.

The Role of Accelerator Chips in LLM Serving

Now that you understand the importance of LLM optimization, the next step is to explore the hardware that powers LLMs. Choosing the right accelerator setup is often one of the most important decisions in serving LLMs, because a system's hardware constraints dictate its memory capacity, computational power, and efficiency. Understanding these factors will help you maximize your model's performance and cost-effectiveness.

In this section, we'll start by breaking down how to read GPU specifications. GPU specs can be overwhelming due to the sheer amount of technical details, but to simplify things, we'll focus on the attributes most relevant to LLMs: GPU compute power, memory capacity, memory bandwidth, and interconnects. Understanding these factors will help you make informed decisions when selecting a GPU for your LLM inference and training.

After covering those key concepts, we'll analyze and compare the specs of several popular NVIDIA GPUs (as of the time of writing, in early 2025).

We'll also share our insights on which GPUs are best suited for different LLM use cases.

We'll end the section by looking at hardware options other than Nvidia chips, such as TPU and NPU.

Reading GPU specs

In this section, we will show you how to read GPU specifications by analyzing several attributes of two Nvidia GPUs: H100 SXM and H100 NVL. Understanding GPU specifications will not only help you choose the best hardware for AI inference, it will also benefit your model training.

Table 2-1. Specification comparison between H100 SXM and H100 NVL. (Source: [NVIDIA](#)).

	H100 SXM	H100 NVL
FP64	34 teraFLOPS	30 teraFLOPS
FP64 Tensor Core	67 teraFLOPS	60 teraFLOPS
FP32	67 teraFLOPS	60 teraFLOPS
TF32 Tensor Core	989 teraFLOPS	835 teraFLOPS
BFLOAT16 Tensor Core	1979 teraFLOPS	1671 teraFLOPS
FP16 Tensor Core	1979 teraFLOPS	1671 teraFLOPS
FP8 Tensor Core	3958 teraFLOPS	3341 teraFLOPS
INT8 Tensor Core	3958 TOPS	3341 TOPS
GPU Memory	80 GB	94GB
GPU Memory Bandwidth	3.35 TB/s	3.9 TB/s

GPU Compute Attribute

GPU compute performance is usually measured in *teraFLOPs*. *Floating-Point Operations Per Second* (FLOPs) are a measure of a chip's theoretical GPU compute capability, based on how many floating-point operations

(like addition and multiplications) it can perform within a second. In other words, it's a measure of how fast it is, theoretically. *Tera-* means 1 trillion, or 1,000,000,000,000. For example, the top seven attributes listed in [Table 2-1](#) FP64, FP64 Tensor Core, FP32, FP32 Tensor Core, BFLOAT16 Tensor Core, FP8 Tensor Core, INT8 Tensor Core – represent the compute performance of different precisions in teraFLOPs.

From the comparison table, you can see that the H100 SXM can perform $1979 * 10^{12}$ operations in a second, giving it FP16 precision! (*FP16* stands for “half-precision floating point,” more often called *16-bit*. You can double that if you can use a model with FP8 precision. FP16 and FP8 (*8-bit*) are lower-precision numerical formats used in deep learning models. Because FP8 is half the size of FP16, computation can be almost twice as fast. (We will discuss this a lot more in Chapter 6.)

Comparing the FLOPS horizontally between two versions, H100 SXM and H100 NVL, the NVL version is slightly less performant in this aspect. But is it really just worse for all use cases? Maybe not. Let's continue our analysis.

GPU Memory Attributes

Next, let's look at two equally important, if not more so, GPU specifications: *memory* (also called *VRAM*), which indicates the memory capacity for loading a model, and *memory bandwidth*, which measures the

data-transfer speed for GPU computation. H100 NVL has a bigger memory size (94GB) than H100 SXM (80GB). What's more, it also has a higher GPU memory bandwidth: 3.9 TB/s (terabytes per second), compared to 3.35 TB/s in SXM.

Now we have a choice to make: H100 SXM or H100 NVL? H100 SXM calculates faster than H100 NVL, since it has higher FLOPS, but H100 NVL has a better memory capacity and bandwidth. How do we choose between the two?

Let's use an analogy here. Imagine the GPU is a big pizza kitchen, and our goal is to prepare pizzas as quickly as possible to serve as many customers as we can. *Compute power*, measured in FLOPS, represents how fast and powerful our oven is, while the *GPU memory bandwidth* determines how efficiently we can prepare all the ingredients and supply the dough to the oven. *GPU memory capacity* is the fridge's capacity to store all the dough.

Do you see the interaction between all three? If we don't have a big enough fridge to hold the dough, customers will now be sitting there hungry—just as, with a small GPU memory, we cannot load the model. But let's suppose our fridge is just big enough to serve all of our customers. Let's look at the oven.

If we have a very advanced oven, but we can only supply dough very slowly, our powerful oven will be running without baking much pizza. We

can't make good use of its power! We'll have a similar problem if we have super high FLOPs for compute, but low GPU memory bandwidth.

Conversely, if the oven isn't very powerful but we can prepare dough very fast, the pizza will take a very long time to bake and customers will have to wait for their food. This is much like having weak compute but high memory bandwidth.

Thus, it's very important to find a balance between compute power, memory size, and bandwidth that fits your workload. We will discuss this more in the next few sections.

GPU Interconnect Attributes

GPU interconnects enable high-speed data transfer between multiple GPUs, both within a single node and across multiple nodes. It used to be crucial only for model training, but GPU interconnect has become more and more important for model serving.

With the emergence of larger and larger models (LLMs), one GPU is no longer powerful enough to host most models. You might need to serve a very large model which needs multiple GPUs, with each loading a portion of the model weights or layers. Or when dealing with a latency-sensitive use case, if one GPU is not powerful enough to achieve the latency requirement, you'll need multiple GPUs to work together to achieve faster inference speed. When you face these situations, the GPU interconnect

specs become critical. In this section, we will discuss GPU interconnect in detail, starting with *intra-node* interconnects (those within a single node) and then moving to *inter-node* interconnects (those across multiple nodes).

Intra-Node Interconnects

To help you understand intra-node interconnects, let’s look at three variations of the Nvidia GPU H100 in [Table 2-2](#).

Table 2-2. Comparing GPU interconnects across three types of Nvidia GPU H100.

	H100 PCIe	H100 NVL	H100 SXM
Form Factor	PCIe	PCIe	SXM
NVLink Support	No (Optional)	NVLink Bridge	NVLink/NVSw
Interconnect	128 GB/s with	600 GB/s with	900 GB/s
GPU-to-GPU bandwidth	PCIe	NVLink Bridge connecting 2 H100 GPUs only	connecting up-to H100 GPUs

The first row of [Table 2-2](#) tells you the form factor of the GPU. *Form factor* defines a GPU’s physical size, power requirements, and

cooling design, which eventually determine how it integrates into a server or workstation. In this example, H100 SXM follows the SXM form factor. SXM-version GPUs are mounted directly onto the motherboard via a custom socket, rather than being inserted into a PCIe slot. This configuration enables faster connections, better power delivery, and enhanced cooling to achieve better performance with intense multi-GPU workloads.

In contrast, both H100 NVL and H100 PCIe follow the Peripheral Component Interconnect Express, or PCIe, form factor. PCIe is a commonly used standard data-transfer connection between various electronics. A PCIe version of a GPU means the GPU is installed into a PCIe slot, making it more widely compatible and cheaper, but less performant.

The next two rows show the NVLink technologies each variant uses and the corresponding intra-node GPU-to-GPU bandwidths. *NVLink* is a Nvidia high-speed interconnect technology that enables direct GPU-to-GPU interconnect for fast, high-bandwidth communication. For the variants without NVLink, GPU-to-GPU communication falls back to PCIe, which is much slower.

Let's look at some GPU interconnect topology diagrams (shown in [Figure 2-4](#) through [Figure 2-7](#)), to understand the difference between

NVLink and PCIe, as well as different configurations of NVLink using these three versions of Nvidia H100 GPUs.

H100 PCIe

In [Figure 2-4](#), two GPUs use PCIe, which has GPU-to-GPU bandwidth of 128 GB/s. (H100 PCIe can be installed with NVLink Bridge but needs to be bought separately.) This configuration is the cheapest. If you do not need fast GPU-to-GPU communication, such as if you have two small models that run independently on one GPU each, a basic PCIe connection is sufficient.

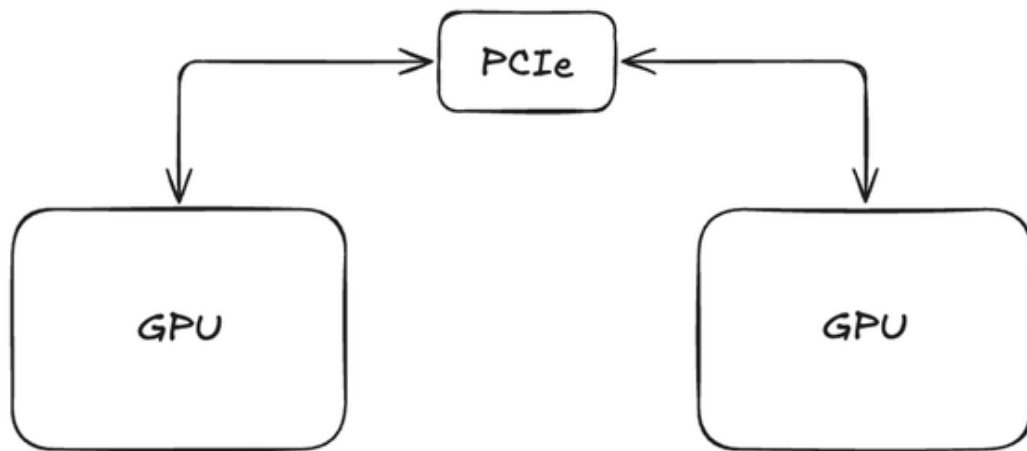


Figure 2-4. GPU interconnect with PCIe.

H100 NVL

H100 NVL also follows the PCIe form factor, but adds NVLink Bridge. This technology connects a pair of H100 NVL interconnects together with a bandwidth speed of 600GB/s, as shown in [Figure 2-5](#).

The drawback of the NVLink Bridge setup is that it can only connect two GPUs. If you need more than two GPUs to work together, the slower PCIe connection is still required. For example, in a four-GPU setup, GPU1 and GPU2 are linked with NVLink Bridge, and GPU3 and GPU4 are also linked with NVLink Bridge. This means that communication between GPU1 and GPU2 takes place at 600 GB/s. The same goes for GPU3 and GPU4. But the connections between GPU1 and GPU3, and between GPU2 and GPU4, are still using PCIe, which is 128 GB/s. This setup is a good middle ground that balances cost and performance.

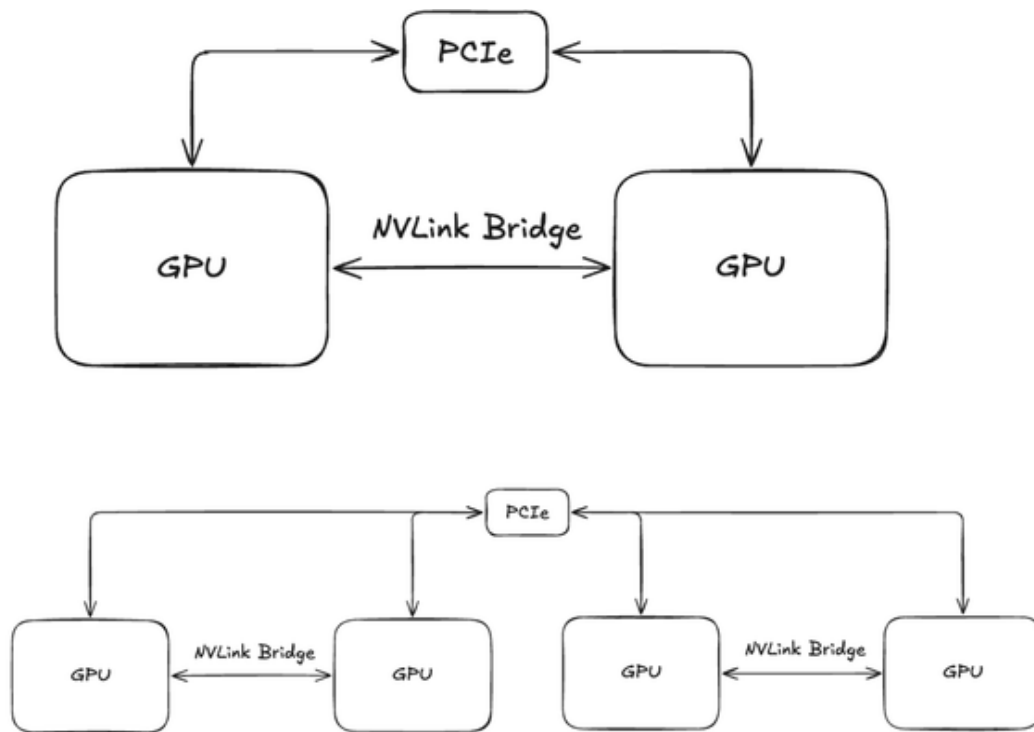


Figure 2-5. Two-GPU (top) and four-GPU (bottom) interconnects with NVLink Bridge.

With the SXM version, up to eight GPUs within the same node can be connected with NVLink, as shown in [Figure 2-6](#).

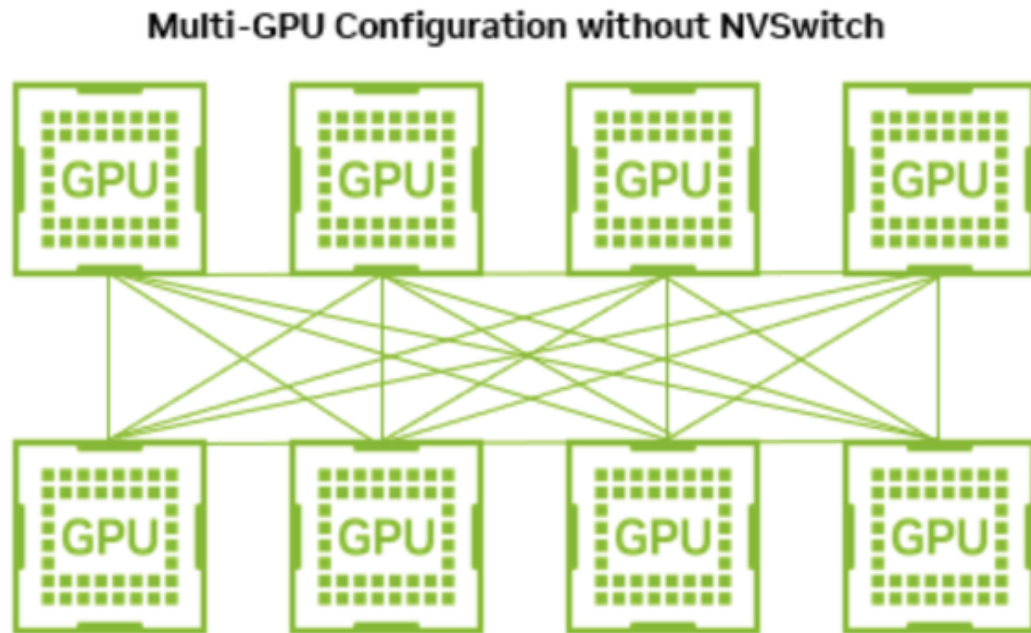


Figure 2-6. GPU interconnect with NVLink. (Source: [NVIDIA Technical Blog](#))

[Figure 2-6](#) shows how GPUs can be connected with NVLink in SXM. Since SXM can connect eight GPUs and each GPU's interconnect bandwidth is 900 GB/s (see [Table 2-2](#)), each GPU must split its 900 GB/s total bandwidth into seven dedicated 128 GB/s (900GB/s divided by 7) point-to-point connections, each connecting to one of the other GPUs in the system.

If we don't like the fact that we are splitting the interconnect bandwidth by the number of GPUs it's connected to, we can use

NVSwitches to achieve full 900 GB/s bandwidth with all-to-all GPU connections. NVSwitch is a separate, expensive, high-bandwidth switch that connects multiple NVLink-enabled GPUs together.

[Figure 2-7](#) shows that, with four NVSwitches added to the configuration in between, SXM can achieve full 900 GB/s bandwidth in all-to-all fashion for the best performance possible.

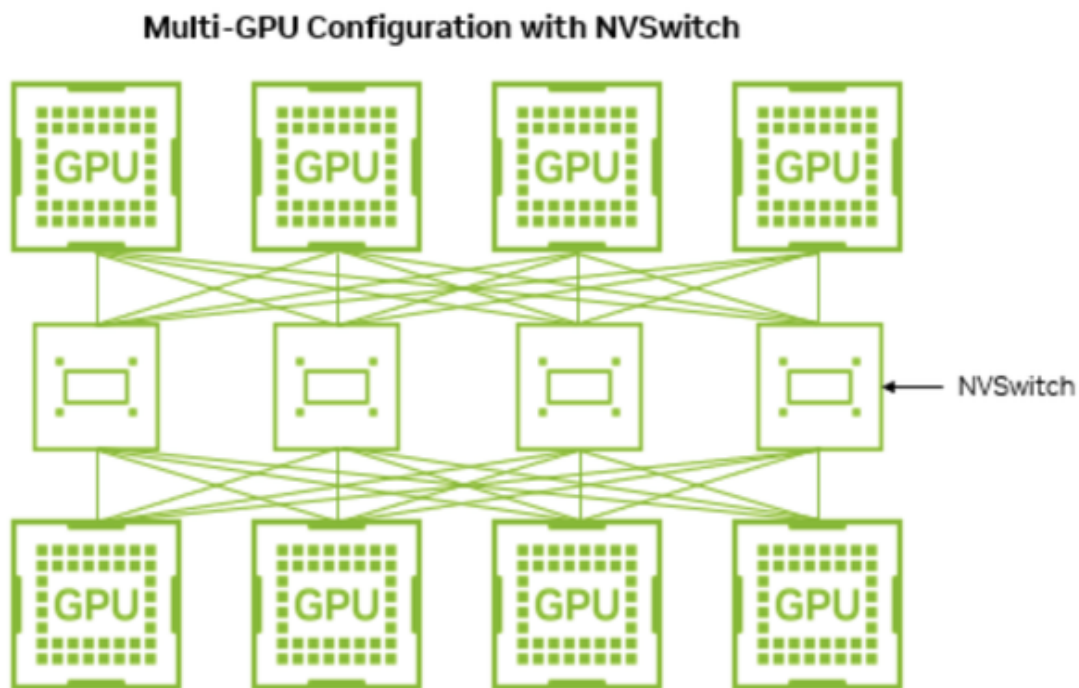


Figure 2-7. GPU interconnects with NVLink and four NVSwitches in between. (Source: [NVIDIA Technical Blog](#)).

Although using an NVSwitch increases interconnect bandwidth greatly, it is the most expensive of all the setups, and can be overkill if your workload does not require that much GPU-to-GPU communication.

Inter-Node Interconnect

All of the setups we just discussed are for intra-node GPU-to-GPU communication. However, it is not possible to have an unlimited number of GPUs in one node. The maximum number of GPUs within a node is usually capped at eight due to physical constraints, power supply, cooling concerns, and software support. As the LLMs become larger and larger, eight GPUs in one node can also become insufficient. In this case, you can shard models across multiple nodes, so even more GPUs can work together to serve the model.

Similar to intra-GPU interconnect, inter-node interconnect becomes important when you're serving models across multiple nodes. One of the best solutions is using InfiniBand(IB) with GPUDirect RDMA. RDMA, which stands for "remote direct memory access," enables direct memory access between GPUs across nodes. NDR 400G InfiniBand, for example, can achieve 50GB/s across nodes—but that's still quite a lot slower than intra-node performance.

See [Table 2-3](#) for a performance comparison of different intra-node vs. inter-node bandwidth. For example, within one node, GPU-to-GPU communication can be 900GB/s with NVLink and NVSwitch while GPU-to-GPU intra-node communication only maxes at 50 GB/s.

Table 2-3. Comparing communication speeds for example H100 setups.

Setup	Bandwidth
GPU-to-GPU within node with NVLink/NVSwitch	900 GB/s
GPU-to-GPU within node with NVLink Bridge	600 GB/s
GPU-to-GPU within node with PCIe	128 GB/s
GPU-to-GPU across node	50 GB/s

For LLM serving, the majority of deployments still run on a single node with one to eight GPUs for each model instance (also called a *replica*). As user traffic increases, we scale up the number of model replicas horizontally, by adding more hardware instances with the same setup. The goal of this approach is to minimize unnecessary internode communication.

In Chapter 6, we will explore how models can be distributed across multiple GPUs and multiple nodes and cover the pros and cons of different setups and techniques, such as Tensor Parallel and Pipeline Parallel.

Recent research is exploring separating the Prefill and Decode phases across different GPUs. Additionally, for large models using a Mixture-of-Experts (MoE) architecture—like DeepSeek V3/R1—techniques like Expert Parallel and Data Parallel are applied even across nodes. We will introduce these advanced topics after covering the foundational concepts.

Comparing the Specs of Popular GPUs

Now that you understand the key GPU specifications, let's explore how to apply this knowledge to determine the best use cases for a given GPU.

Table 2-4. Comparing the specs and costs of popular GPUs. (These costs were current as of spring 2020)

	H200 SXM	H100 SXM	A100 SXM	
GPU Memory Size (GB)	141	80	80	4
FP16 and BF16 TeraFLOPs (TensorCore)	1979	1979	312	3
GPU Memory Bandwidth (TB/s)	4.8	3.35	1.935	0
Support for FP8	Yes	Yes	No	7
NVLink/NVSwitch	Yes	Yes	Yes	1
Cost on-demand hourly on AWS us-west1 or us-west2 (8 chips per instance)	P5e.48xlarge Not available, only reserved	P5.48xlarge ~\$123	P4de.24xlarge \$41 (80GB)	0

[Table 2-4](#) compares the specs and cost of some popular Nvidia chips for inference. In general, more powerful GPUs will always be more expensive, so your selection really depends on whether the model to be served can

benefit from the GPU's increased performance or the specific features it provides. Let's look at some examples:

Small model

If you are serving a fine-tuned model such as Llama-3-8B for a specific use case and latency is not very critical, then A10 might be a decent choice. It already has enough GPU memory to load and execute the model, it's the cheapest option here, and it's highly available, being an older-generation chip.

Mid-sized model

The deepseek-ai/DeepSeek-R1-Distill-Qwen-14B model, which is a 14B model, has almost twice as many parameters as Llama-3-8B. If you're serving this model, stepping up to L40S might be a good idea. Especially considering their support of FP8 precision, L40S can be highly capable for this task. This setup avoids all the overhead of serving models across multiple GPUs; at the same time, it has a good amount of memory left over to achieve long input context, long output generation, and high batch size.

Large model

If you need to serve deepseek-ai/DeepSeek-R1, NVLink Switch becomes a crucial feature to have. This model has 671B parameters, making it too big to fit and fast for even a single H200 to handle. A

likely configuration would be using one machine with eight H200 GPUs connected together in FP8 precision. (The DeepSeek model is that it uses a Mixture-of-Experts (MoE) architecture. This enables further optimizations, such as having experts spread across multiple GPUs or even different nodes. We'll discuss this in the final chapters of the book.)

These are just some simple examples of how different models run on different GPUs. We've introduced quite a lot of new concepts and calculations here, without expanding on them, but don't worry. We will dissect them all in this book, and teach you techniques for finding the best solution that meets your serving requirement and reduces the cost-to-serve.

Please note that, in this hot field, technology changes fast. The latest, hottest chip that everyone wants today can be replaced by a newer one tomorrow, but the foundational concepts you're learning and the intuition you're building for weighing your options do not change.

Other AI Accelerators

So far, we have been solely investigating Nvidia GPUs. However, in the LLM Inferencing space, a lot of other chips are competing with Nvidia, and some are designed specifically for model inferencing. GPUs such as AMD's MI300X and Intel's Gaudi2 are both alternatives to H100 and A100. Tech giants Google's TPU and Amazon's Inferentia are mostly only available on

their respective clouds. Huawei's Ascend NPU is another notable competitor, particularly in markets where Nvidia faces restrictions. And there are many startups in this space, such as Groq, Cerebras, Untether AI, SambaNova, and d-Matrix.

As we write this in spring 2025, Nvidia's General-Purpose Graphics Processing Unit (GPGPU) still dominates this market. There are several reasons for its dominance:

- Some of these offerings are specifically tailored for deep-learning-based model inference, matrix multiplication workloads, or Transformers. Their architectures are simpler, more cost-efficient, and more energy-efficient, but compared to GPGPU, they often struggle with the rapidly evolving demand from model architectures.
- At the software level, Nvidia GPUs have much more mature support with the CUDA ecosystem, which is widely used in both training and inference. Custom chips usually require their own proprietary software stacks, such as ROCm for AMD GPUs, JAX for TPU, and Neuron for Inferentia. Switching over can be costly, especially given the lower levels of community support,
- Some chips that leverage on-chip SRAM to achieve impressive performance with superb latency. However, so far, this often isn't a very cost-effective solution, given the high price of SRAM and the number of chips needed to serve one model instance.

- Nvidia GPUs still have the edge in flexibility to meet different inference setups. They support different levels of precision, such as FP8, high memory bandwidth, and advanced GPU interconnect capability, which are all contributing factors in this area.

While all these factors contribute to Nvidia's continued dominance in the market, the broader landscape of hardware acceleration is evolving rapidly. As new architectures emerge, one of the biggest challenges is overcoming bottlenecks, since data movement has not kept pace with improvements in raw compute power.

Figure 2-8 compares the increases in hardware compute FLOPS, memory bandwidth, and inter-GPU bandwidth over the last 20 years. You can see that the compute capability has improved a lot over time, and at a much faster pace than data movement speed, in both memory bandwidth and inter-GPU bandwidth. Nowadays, GPU data-movement limitations have become an obstacle to utilizing the fast advancements in compute. This problem is usually referred to as the “memory wall.” Quite a lot of accelerator chip manufacturers are trying to tackle this issue.

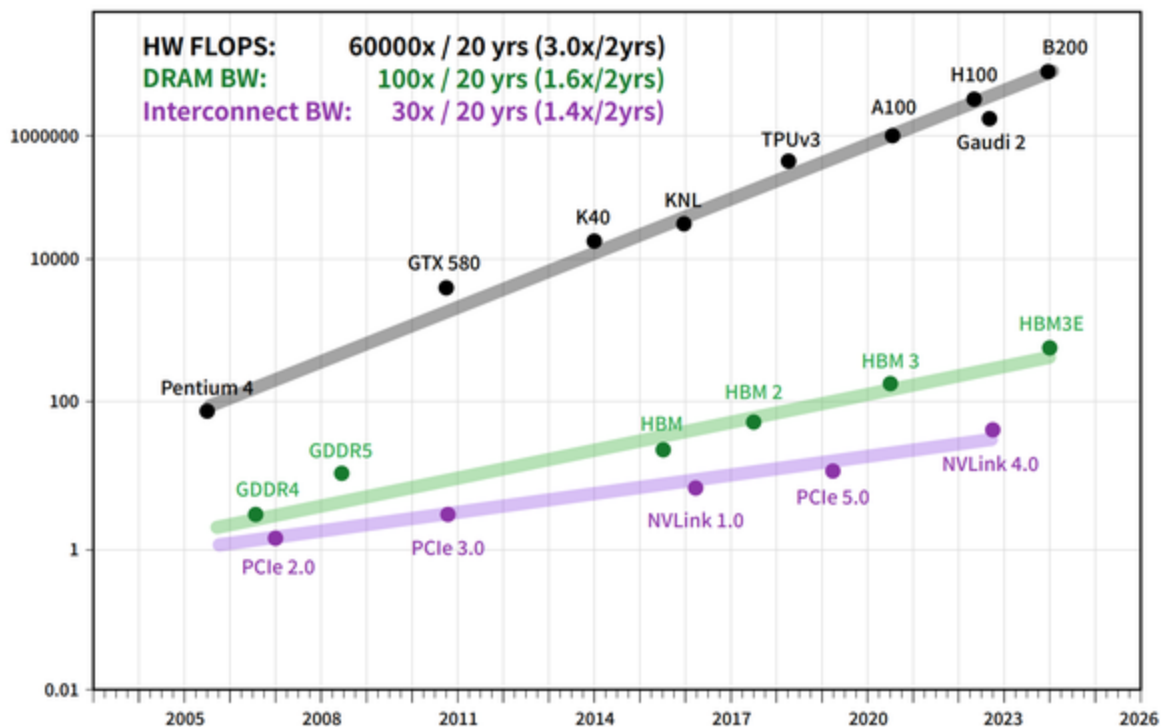


Figure 2-8. Scaling in peak hardware FLOPS and memory and interconnect bandwidth. (Source: [An et al., 2024](#))

Now that you have a solid understanding of GPU specifications, it's time to explore the limitations and bottlenecks you may encounter when actually serving LLMs on GPUs. Identifying these bottlenecks is critical, since the optimization techniques introduced throughout the rest of the book are specifically designed to overcome these challenges and improve efficiency.

Bottlenecks in LLM Model Loading

In the following two sections, we will put LLMs into the mix and see how they interact with the GPU specs you learned about in the two previous

sections. We will start with model loading, then discuss model execution phases.

We'll look at why models need to be hosted in GPU memory, identify the major consumers of GPU memory, and walk through how to estimate GPU memory requirements for a given model with a given workload.

The Model Loading Process

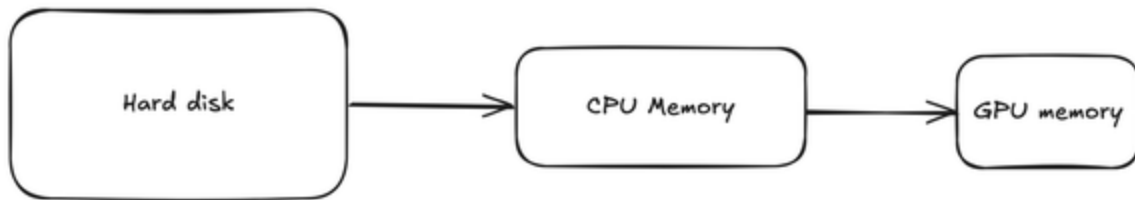


Figure 2-9. Data flow during model-weights loading.

The first thing you need to do for efficient LLM serving is to load the model weights and cache them in GPU memory. During the loading process, the model weights are first moved into CPU memory (also called *system memory*) and then transferred to GPU memory, as shown in [Figure 2-9](#). Once loaded, the weights remain cached in GPU memory, ready to serve incoming requests. Thus, it's essential to have a big enough GPU memory size to hold the model weights.

Some of you may wonder: what if we cached the model in a bigger CPU memory, or just load it on-demand when serving requests come in? The answer lies in data transfer speeds. Both hard disks and CPU memory are

significantly slower at moving model weights, making them impractical for real-time inference. [Table 2-5](#) compares the data-transfer bandwidths of SSD, CPU memory, and GPU memory. Loading from the hard disk would be even slower.

Table 2-5. Comparing data-transfer bandwidth between SSD, CPU memory, and GPU memory.

Hard Disk (SSD) bandwidth	CPU memory bandwidth	GPU memory bandwidth
0.5 to 14 GB/s	50 to 200 GB/s	300 GB/s to 3 TB/s

It’s crucial to distinguish between CPU (system) memory and GPU memory. The GPU memory is where the model weights should be cached. If model weights are stored inside CPU memory, they need to be transferred to GPU memory to utilize the GPU compute. This extra step introduces an unacceptable delay in serving responses.

Estimating Model Size

How much GPU memory will a large AI model require? When calculating the memory footprint of a model, there are two key variables: the model’s parameter count and the data type (precision) of its parameters.

Let’s take a model from HuggingFace as an example. A model’s number of parameters can often be found directly in its name. For instance, the name

[Llama-2-7b](#) indicates that this model contains approximately 7 billion parameters in its model weight.

The data type can be found in the *config.json* file in the model's Hugging Face repository (shown in [Figure 2-10](#)). Specifically, the `torch_dtype` attribute reveals the precision used for model weights. For example, if the value is `float16` (FP16), this means the model uses 16-bit floating-point precision for storage and computation.

Here is an example of a *config.json* file in HuggingFace, which contains the model configuration and information to correctly instantiate and use the model:

```
{
  "_name_or_path": "meta-llama/Llama-2-7b-chat-hf",
  "architectures": [
    "LlamaForCausalLM"
  ],
  "bos_token_id": 1,
  "eos_token_id": 2,
  "hidden_act": "silu",
  "hidden_size": 4096,
  "initializer_range": 0.02,
  "intermediate_size": 11008,
  "max_position_embeddings": 4096,
  "model_type": "llama",
```

```
"num_attention_heads": 32,  
"num_hidden_layers": 32,  
"num_key_value_heads": 32,  
"pretraining_tp": 1,  
"rms_norm_eps": 1e-05,  
"rope_scaling": null,  
"tie_word_embeddings": false,  
"torch_dtype": "float16",  
"transformers_version": "4.32.0.dev0",  
"use_cache": true,  
"vocab_size": 32000  
}
```

The *data type*, or *precision*, of a model refers to the number of bits required to store a single parameter and their corresponding memory sizes. Data type significantly impacts both model size and model performance, because it determines how the model weights are stored. In general, lowering precision—for example, from FP32 to FP16, and then to INT8 or FP8—comes at the cost of some accuracy, but reduces model size and improves serving performance. We will dive deeper in the next chapter; for now, you just need to understand the relationship between precision levels, as shown in [Table 2-6](#).

Table 2-6. Parameter precision and memory size.

	Bits	Size in bytes
FP32 (Single Precision)	32	4
FP16 (Half Precision) / BF16	16	2
INT8 (Quarter Precision) / FP8	8	1

Now we know that the [Llama-2-7b](#) model has around 7 billion parameters that are stored in BF16 format, which means each parameter occupies 2 bytes:

$$\sim 7 \text{ billion parameters} * 2 \text{ bytes/parameter} = 14 \text{ billion bytes} = 14 \text{ GB}$$

Looking at the files for the model confirms this estimate. The screenshot in [Figure 2-10](#) shows the actual sizes of *pytorch_model*.bin* files, which sum up to around 13.GB.

 .gitattributes	 Safe	1.52 kB	
 LICENSE.txt	 Safe	7.02 kB	
 README.md	 Safe	22.3 kB	
 USE_POLICY.md	 Safe	4.77 kB	
 config.json	 Safe	614 Bytes	
 generation_config.json	 Safe	188 Bytes	
 model-00001-of-00002.safetensors	 Safe 	9.98 GB  LFS	
 model-00002-of-00002.safetensors	 Safe 	3.5 GB  LFS	
 model.safetensors.index.json	 Safe 	26.8 kB	
 pytorch_model-00001-of-00002.bin	 Safe  pickle	9.98 GB  LFS	
 pytorch_model-00002-of-00002.bin	 Safe  pickle	3.5 GB  LFS	
 pytorch_model.bin.index.json	 Safe	26.8 kB	
 special_tokens_map.json	 Safe	414 Bytes	
 tokenizer.json	 Safe	1.84 MB	
 tokenizer.model	 Safe	500 kB  LFS	
 tokenizer_config.json	 Safe	1.62 kB	

Figure 2-10. Llama-2-7B model files in the [HuggingFace model repository](#).

Estimating KV Cache Size

Continuing the example in the last section, we now know the model size, which is around 14GB. Suppose you have a GPU with 16GB of memory. You should be able to load it fine, since 16 is more than 14. It will very

likely also run fine when you send in one short single request. Yay! But hang on. A GPU with *just enough* memory to hold the model is not ideal.

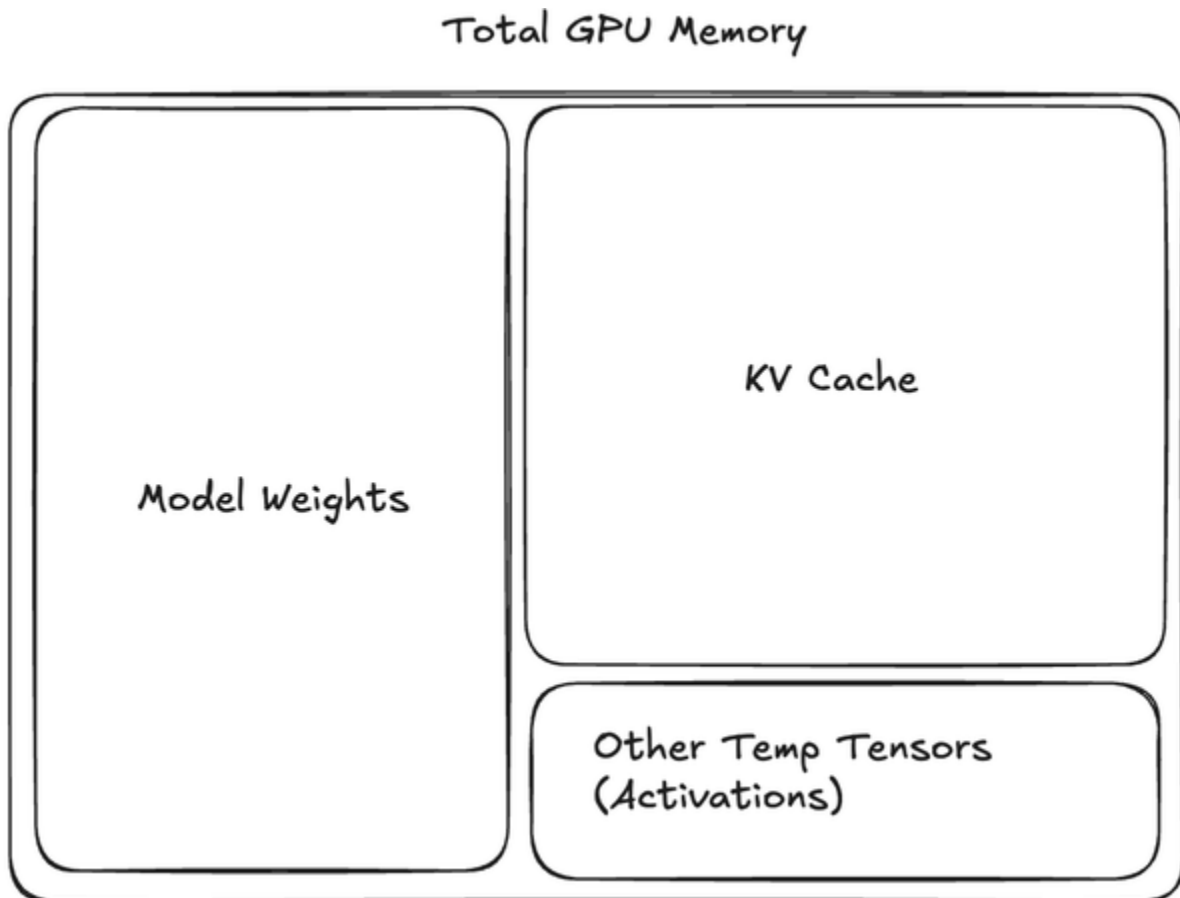


Figure 2-11. GPU memory usage breakdown.

Recall that in Chapter 2, we dived deep into KV caching, which is an optimization technique that trades some amount of GPU memory space for much faster serving performance by caching the intermediate results. As it decodes the next token, the model can reuse those results without redoing the computation. [Figure 2-11](#) shows an example of how model weight and KV cache co-locate inside GPU memory. A small amount of extra memory

should be set aside for intermediate computations. If you only leave a tiny amount of GPU memory for the KV cache, it constrains you to run at very limited batch sizes and with short context. Now let's look at why, and see how to estimate how much KV cache space to reserve.

The formula for estimating KV cache size for each token is:

KV cache size per token = 2 x Number of Layers x Number of Attention Heads x Head Dimension x Data Type Size

Next, we need to account for the effects of the model's architecture and size. In this simple example, we will use Llama-2-7b, which is still using the basic form of *multi-head attention* (MHA). In later chapters, we will introduce other novel ideas to reduce and compress KV caches from multi-query attention (MQA), grouped-query attention (GQA), and multi-head latent attention (MLA), which was introduced in DeepSeek V2 and is used in DeepSeek V3 and R1.

Llama-2-7b has 32 attention-head layers and 32 attention heads, with a head dimension of 128 (4096/32). Suppose we still use half precision. Referring to [Table 2-6](#), you can see that the precision comes to 2 bytes per token.

Using the formula above, we get:

KV cache size per token = 2 x 32 x 32 x 128 x 2 = 524,288 bytes = 0.5 MB

We now know that each token requires 0.5MB. The next question is: how many tokens do we need to cache? The maximum number of tokens can be calculated as the maximum batch size times the maximum sequence length. *Max batch size* determines how many parallel users or /requests one model instance can process at a time. *Max sequence length* limits the total of input length and output length. Thus, if we are processing more requests together and dealing with longer prompts and/or longer generation, the KV cache size will add up:

Total KV cache = KV cache size per token x max number of tokens in cache

= KV cache size per token x (max batch size x max sequence length)

Suppose we use the model to summarize long paragraphs, using a max sequence length of 4,096. With a batch size of 16, the calculation would be:

Total KV cache = 0.5 MB/token * 4,096 tokens per request * 16 requests = 32GB

That's even bigger than the model itself, which is now 14GB!

Now let's consider two chips, with 24GB and 48GB GPU memory, respectively, still with the max sequence length of 4,096. As shown in [Table 2-7](#), using total GPU memory size and subtracting the model size, which is around 14GB, we can calculate the estimated memory left. We can

then derive the batch size by dividing the product of the sequence length (which is 4,096) and 0.5MB per token. In this case, the A10 GPU can only serve 4 parallel requests, while L40S can serve 16. Even though L40S is more expensive, it still comes out to be more cost-efficient. (Note, however, that we usually cannot achieve the maximum theoretical batch size from the batch size calculation, because we also need to reserve space inside the GPU for *activations*, which are the tensors created in the intermediate steps.)

Table 2-7. Batch size calculation and cost comparison between two different GPUs.

GPU and Memory Size	Memory Left		On-demand hourly cost per chip from AWS
	After Model Weight Loaded	Max Possible Batch Size	
A10 24GB	10GB (24 - 14)	4 ($10 \times 1024 / (0.5 \times 4096)$ = 5)	\$2
L40S 48GB	34GB (48 - 14)	16 ($34 \times 1024 / (0.5 \times 4096)$ = 17)	\$3.75

So far, you’ve learned that during model loading, we want to reserve enough space for both the model weights and KV cache. Model weights

need to be cached inside the GPU memory; KV cache space is reserved for execution. [Figure 2-12](#) shows an example of how GPU memory usage changes from idling to execution. During execution, KV cache size will continue to grow as the sequence length grows and we cache more tokens. What we need to make sure is we have enough memory beyond the peak memory usage at the very end of the generation to avoid Out-of-Memory (OOM) errors.

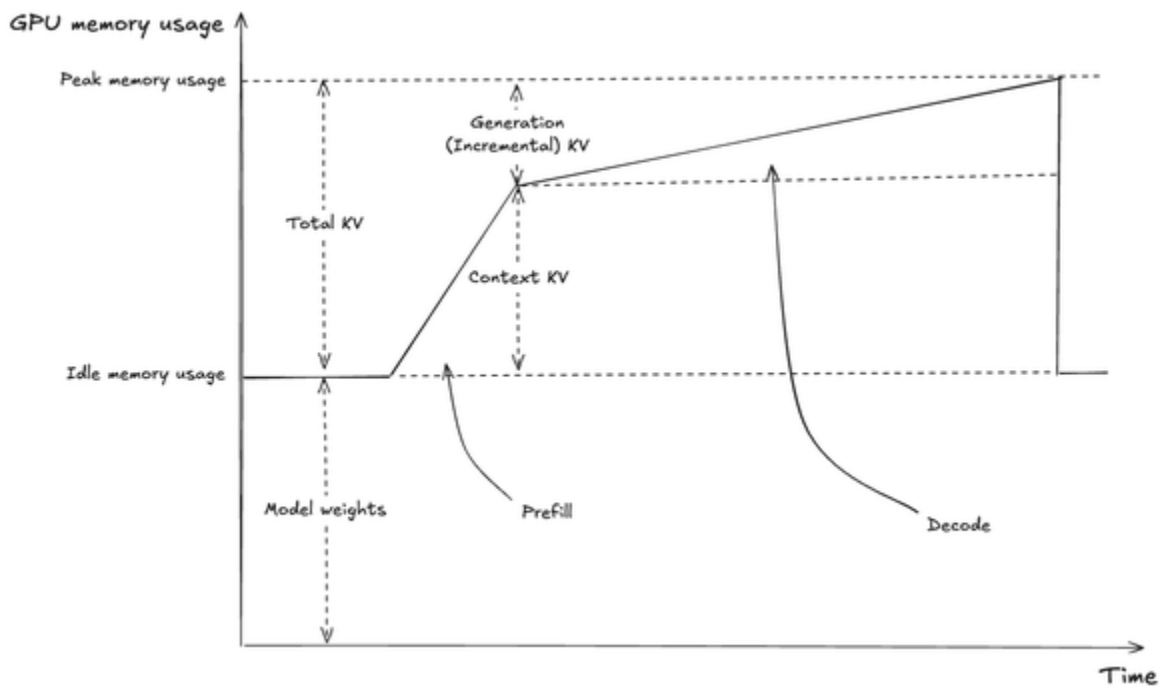


Figure 2-12. How GPU memory usage changes from idling to execution.

In later chapters, we will show you techniques (such as prefix caching) that may require more space to achieve faster inference, especially a lightning-fast *time to first token* (TTFT). But as a general rule of thumb, when estimating model GPU memory requirements, we recommend getting

started with GPU memory approximately twice the model size to achieve better parallelism and unleash more GPU performance.

Bottlenecks in LLM Model Execution

LLM execution is a complex process. Recall what you learned in Chapter 2: when an LLM processes input and then generates output, it goes through two distinct phases: Prefill and Decode. These phases exhibit significantly different GPU utilization patterns. To analyze this, we will introduce the concepts of *arithmetic intensity*, which measures computational efficiency, and the *roofline model*, which illustrates hardware limitations and performance constraints.

Boundaries of GPU Compute and Memory Bandwidth

With model loading and its relationship with model size out of the way, now we have the model entirely loaded inside GPU memory and ready for model serving. It is time to discuss the elephant in the room: is the model serving bounded by GPU compute FLOPs or GPU memory bandwidth? In other words, recalling the pizza kitchen analogy, do we need a bigger oven that can bake faster, or do we need to prepare the ingredients and dough faster? Which of the two is the bottleneck in our model serving workload?

To analyze that, we need the concept of *arithmetic intensity*: the ratio of algorithm implementation operations to the number of bytes accessed. This basically calculates the compute (FLOPS) over data movement to the compute unit in a ratio of FLOPS per byte.

$$\text{Arithmetic Intensity} = \text{Number of FLOPS} / \text{Data Movement}$$

If a workload requires very low computation but needs a lot of data reading and writing, then it has a low arithmetic intensity. On the contrary, if a workload does not read and write a lot of data but performs a lot of computations on a small amount of data, it has a high arithmetic intensity.

Now, let's look at data movement, which is often confused with model loading. *Data movement* happens at model execution time, when the model weights are already in GPU memory. During serving, the model weights are moved from GPU memory all the way to the register, so the actual compute can execute the operation ([Figure 2-13](#)).

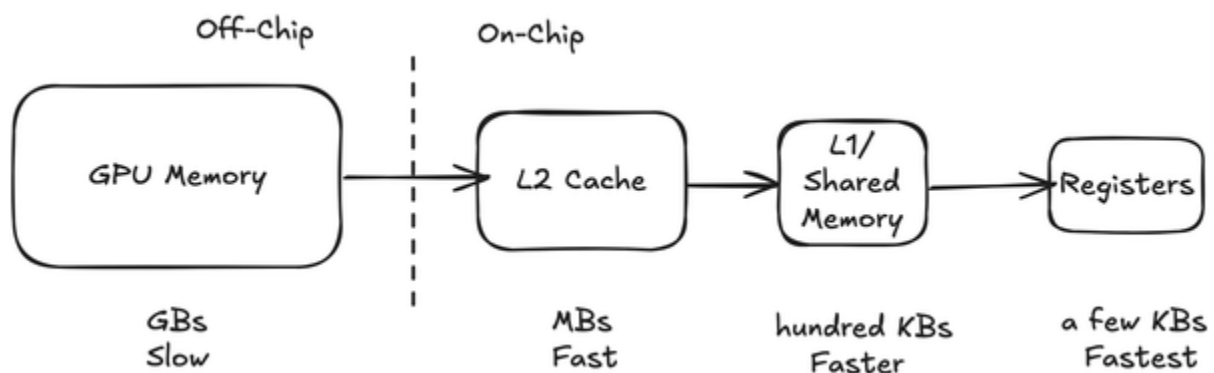


Figure 2-13. Data movement during model serving, as model weights are loaded to register for computation.

During model serving, the model weights stored inside GPU memory are constantly read, along with all intermediate step outputs to registers for computation. Since the GPU memory is the slowest of the bunch, GPU memory bandwidth is the number we should use for the data movement to the compute unit. (It's a little different if the weights or outputs are small enough to fit into the on-chip caches in a smarter way, such as flash attention, but we'll talk about that in Chapter 6.)

Just by looking at the GPU specs, such as the L40S GPU specs in [Table 2-8](#), we can calculate the chip's theoretical arithmetic intensity.

Table 2-8. Specifications for the Nvidia L40S GPU.

GPU Memory	48GB GDDR6 with ECC
Memory Bandwidth	864 GB/s
Interconnect Interface	PCIe Gen4 x16: 64 GB/s bidirectional
NVIDIA Ada Lovelace Architecture CUDA Cores	18,176
NVIDIA Third-Generation RT Cores	142
NVIDIA Fourth-Generation Tensor Cores	568
RT Core Performance TFLOPS	212
FP32 TFLOPS	91.6
TF32 Tensor Core TFLOPS	183
BFLOAT16 Tensor Core TFLOPS	362.05
FP16 Tensor Core	362.05
FP8 Tensor Core	733
Peak INT8 Tensor TOPS	733

For half precision (FP16 Tensor Core), we can calculate the theoretical arithmetic intensity for the chip:

$$362 \text{ TeraFLOPs} / 864 \text{ GB/s} = (362 * 10^{12} \text{ FLOPs}) / (864 * 10^9 \text{ Bytes}) \approx 419 \text{ FLOPs/B}$$

Now that we have the arithmetic intensity, we can draw a *roofline model*, a visual performance model that relates a system's compute capability and memory bandwidth to help identify whether an application is compute-bound or memory-bound. A *naive roofline model* for the L40S chip, a simplified version that estimates a system's achievable performance based only on its peak compute throughput and memory bandwidth, assuming ideal conditions without accounting for hardware complexities, is shown in [Figure 2-14](#).

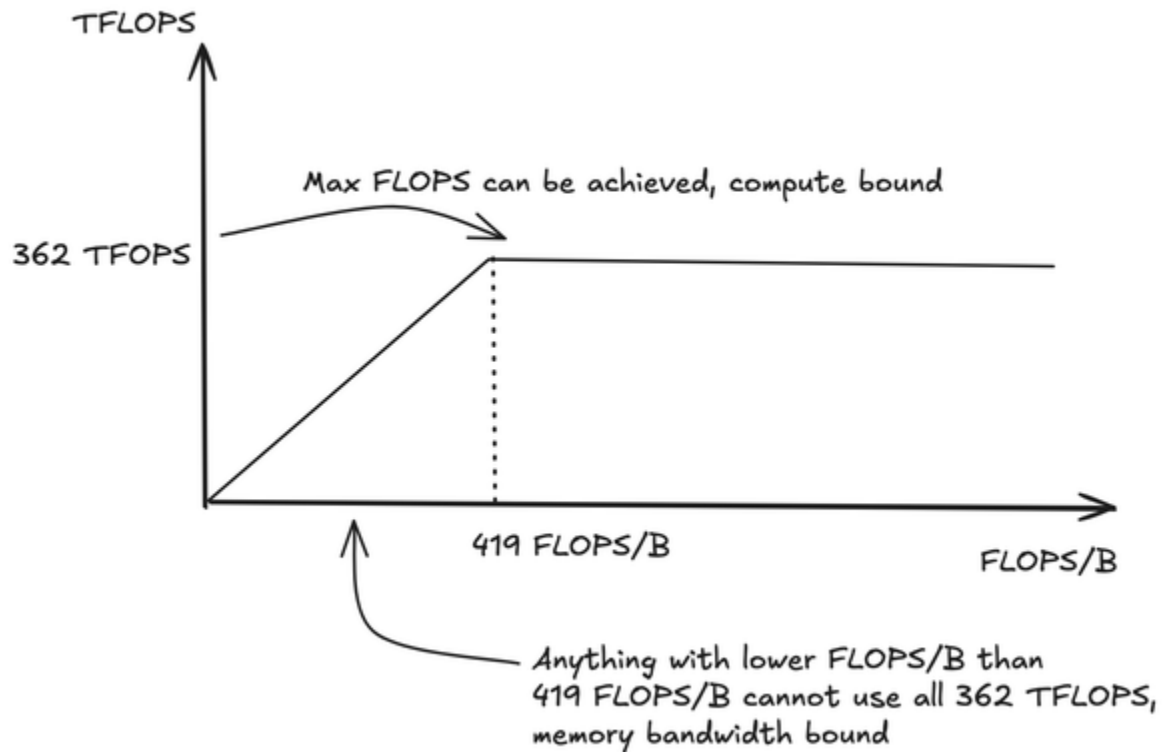


Figure 2-14. Roofline model for the L40s chip.

The x-axis shows the arithmetic intensity in FLOPS/B while the y-axis shows teraFLOPs (TFLOPS). [Figure 2-14](#) shows how much compute the GPU can theoretically do given a certain arithmetic intensity in FLOPS/B. If the arithmetic intensity is lower than the 419 FLOPS/B we calculated, then we cannot use all the 362 TFLOPS the GPU is capable of.

It's time to try using arithmetic intensity and a roofline model to analyze whether a specific workload on a given GPU is bounded by compute or memory bandwidth. Let's say we have a workload to run with ~210 FLOPS/B, basically half of the crossover point of 419 FLOPS/B. This is depicted in [Figure 2-15](#), data point 1. In data point 2, where the workload

has 1,000 FLOPS/B, which easily hits the maximum TFLOPS the chip can do. In this case, data point 1 is bound by memory bandwidth, because it cannot utilize the extra compute it has available. (We still have a lot of space in the pizza oven, but we can't supply the dough fast enough.) Data point 2 is considered to be compute-bound because we've already reached the maximum compute available. No matter how much more data we can read, this is the fastest we can calculate using this chip. (We're supplying dough very fast, but the oven has limited capacity and we have to wait for its current load to finish baking.)

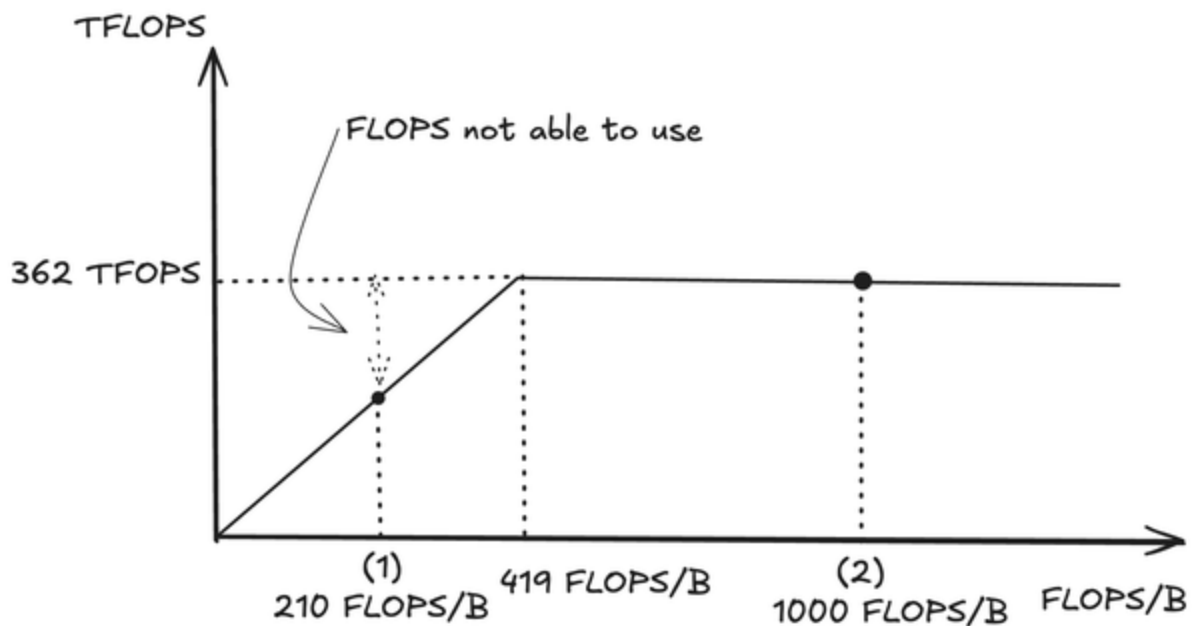


Figure 2-15. Analyzing a workload and how it is bounded using arithmetic intensity and a roofline model.

Now, with a good understanding of the hardware limitations, we need to figure out what an LLM serving workload looks like. Is it memory

bandwidth-bound with low arithmetic intensity, or compute bound with high arithmetic intensity?

Arithmetic Intensity in Matrix Multiplications

To understand whether the LLM serving workload is compute-bound or memory bandwidth -bound, we need to calculate the arithmetic intensity of the various layers inside the LLM's architecture.

In Chapter 2, you learned that an LLM is mostly made up of Transformer blocks. Inside the Transformer blocks, there are two main components: self-attention layers and feedforward layers. The vast majority of computations for both are matrix multiplications. In LLM architectures, there are many other layers, such as element-wise operations and reduction operations, all of which typically have low arithmetic intensity because the computation on each data point loaded is not as high as in matrix multiplications. But these layers only represent a small portion of the overall computation. So, in this section, we'll first show you how to estimate the arithmetic intensity of a matrix multiplication operation (often shortened to *matmul*).

Matrix multiplication means calculating dot products between rows of the first matrix and columns of the second matrix to produce a new matrix.

[Figure 2-16](#) shows an input matrix with shape $[M, K]$ doing a matmul with a weight matrix $[K, N]$, yielding the output matrix $[M, N]$ shown on the right.

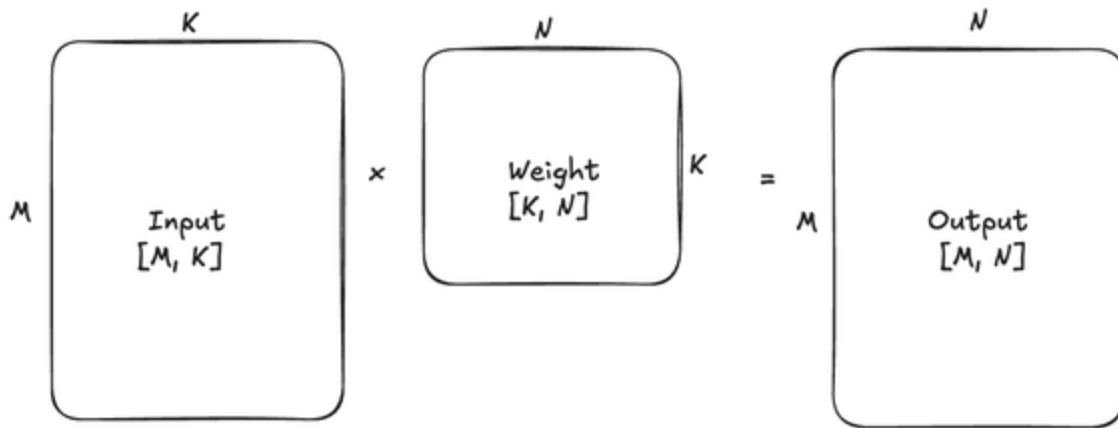


Figure 2-16. Generic matrix multiplication of input and weight matrices, producing an output matrix.

The naive code to calculate the matmul is shown below in nested triple `for` loops:

```
for m in [0, M):
    for n in [0, N):
        for k in [0, K):
            Outputs[m][n] += Inputs[m]
```

To calculate the arithmetic intensity, we need to calculate two things: the *numerator*, which is the number of operations performed during the matmul, and the *denominator*, which is how much data is transferred.

Numerator

For number of operations, based on the code snippet above, there are $M * N * K$ multiplications and $M * N * (K - 1)$ additions. We can

simplify it to

$$\# \text{ of Operations} = 2 * M * N * K$$

Denominator

For data movement, both input matrices are needed to be read and output matrix is needed to be written. Assuming the precision of the values inside the matrices are all half precision, which has size of 2 bytes each. We can get

$$\text{Data Movement} = 2 * (\text{Inputs size} + \text{Weight size} + \text{Outputs size})$$

$$= 2 * (M * K + K * N + M * N)$$

Thus:

$$\text{Arithmetic Intensity} = \# \text{ of Operations} / \text{Data Movement}$$

$$= 2 * M * N * K / (2 * (M * K + K * N + M * N))$$

$$= M * N * K / M * K + K * N + M * N$$

Using our formula for calculating the arithmetic intensity of a matmul, let's see how the arithmetic intensity changes given different sizes of matrices. Supposing for simplicity that $M = N = K$, let's calculate a few values for arithmetic intensity, as shown in [Table 2-9](#).

Table 2-9. Calculating arithmetic intensity values for matrices of different sizes.

Matrix Size (M=N=K)	Arithmetic Intensity (FLOPs/Byte)	Verdict based on L40S
64	21	Memory Bandwidth Bound
512	170	Memory Bandwidth Bound
4096	1365	Compute Bound

The operation can be pushed to compute bound if the matrices in the matrix multiplication operation are large enough. Then the question in LLM serving is: Are the matrices large enough? You might think, given the size of the model, it should be, right? Not always. Let’s break that down in the next section.

Applying arithmetic intensity analysis to the LLM Prefill and Decode phases

You now know that, the bigger the compute matrix, the higher the computational intensity is during matrix multiplication. However, it’s not always possible to achieve big matrix multiplications in LLM serving. This is especially true when the request batch size is 1, which means the model is

only processing a single request at the time. (We will discuss batching in Chapter 6.)

Let's dive deeper into the shape of the input data passed to the model. The "shape" here describes the dimensions of the data, commonly referred to as its *tensor*. In this case, our input tensor is three-dimensional and has a shape of [batch size, sequence length, model hidden dimension]. *Batch size* is the number of requests we are processing at once. *Sequence length* is the number of tokens in the input prompt. From simple phrases such as "Hello" to big paragraphs a million tokens long, the sequence length is highly variable and mostly up to the user. *Model hidden dimension* is the number of values each token's vector has in the model, impacting its capacity and performance. Since we are now only considering one request, batch size is set to 1. Consequently, the three-dimensional input tensor can be reduced to a matrix that has only two dimensions [sequence length, model hidden dimension].

Before we start the arithmetic intensity analysis, let's revisit the two main phases of model serving from Chapter 2: Prefill and Decode. The *Prefill* phase is the initial processing stage, where the model encodes the input prompt before generating any output tokens. During this phase, the model does two things. First, it computes attention over the input tokens using its transformer layers; second, it generates the first set of KV cache entries and stores them for the subsequent generation phase.

In the next stage, the *Decode* phase, the model generates tokens one by one using previously computed KV cache.

Now let's analyze the arithmetic intensity of the Prefill and Decode phases by looking at their matrix multiplication. You will see the difference in [Figure 2-17](#).

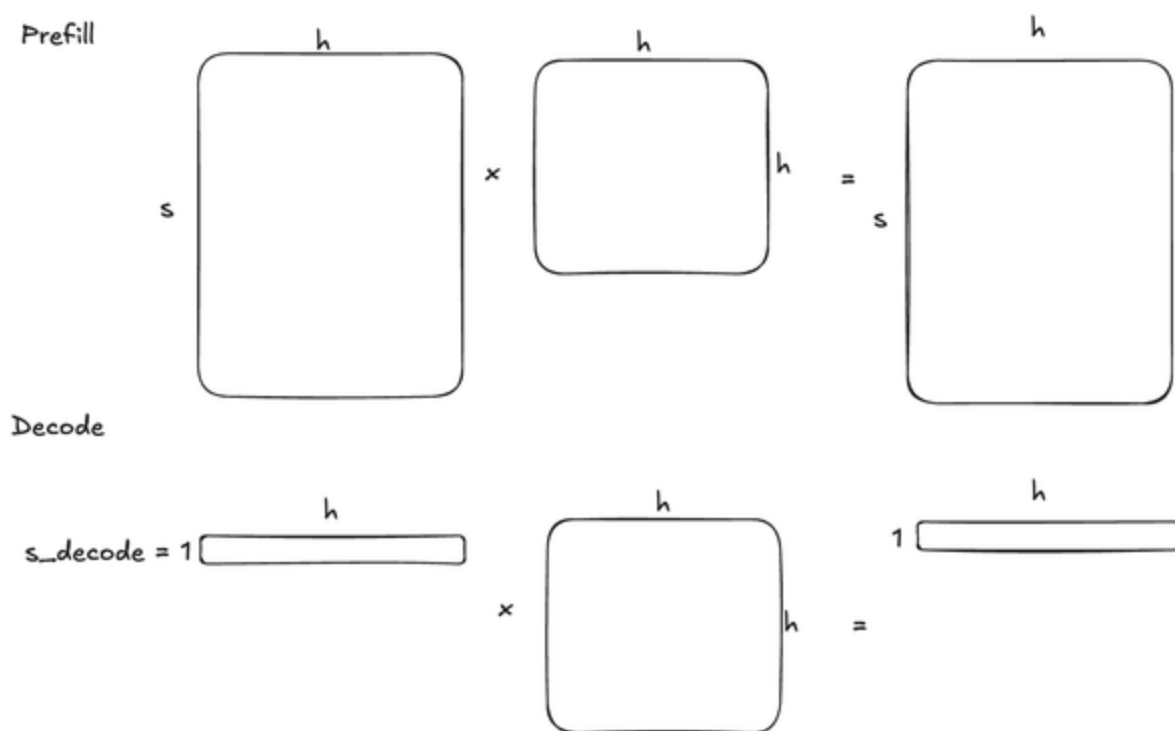


Figure 2-17. Input size difference between the Prefill and Decode stages in a matmul operation.

In [Figure 2-17](#), M , N , and K —which we used in [Figure 2-16](#) for generic matrix multiplication—are replaced with h and s to demonstrate the actual shape of the matrices in LLM serving. The variable h corresponds to the weight (or “hidden”) dimension, which does not change between Prefill and Decode. In Prefill, because all tokens are processed together, the matrix has

a large dimension for the number of rows in s , which is the sequence length of the input. During Decode, however, because tokens are generated one token at a time, s_decode , which denotes the sequence length during decode, is only 1.

Next, in [Table 2-10](#), let's plug the numbers back into the arithmetic intensity formula, which again is:

$$\text{Arithmetic Intensity} = M * N * K / M * K + K * N + M * N$$

$$= s * h * h / s * h + h * h + h * h$$

Table 2-10. Arithmetic intensity comparison between Prefill and Decode at different sequence lengths

Sequence				
Length		Prefill	Decode	
M = s for	Hidden Dim	Arithmetic	Arithmetic	Verd
Prefill	K = N = h	Intensity	Intensity	on L
M = 1 for		(FLOPs/Byte)	(FLOPs/Byte)	
Decode				
64	4096	31	0.5	Mem band boun Prefi Deco
512	4096	240	0.5	Mem band boun Prefi Deco

4096	4096	1365	0.5	Compu
				boun
				Prefi
				Mem
				band
				boun
				Deco

From the numbers we get from for Prefill and Decode calculations, you can see that Prefill can have a high enough arithmetic intensity to saturate GPU compute, making Prefill compute bound when the sequence length is high. However, no matter how long the sequence is, Decode will always be very low, making it memory bandwidth bound.

In summary, different components such as Prefill vs Decode, and different layers of the model can all have very different arithmetic intensity. Arithmetic intensity is also affected by sequence length (as well as batch size, which we've intentionally skipped for now). All these analyses and calculations are designed to help you develop your intuition about bottlenecks in the different phases of LLM serving, so you'll later be able to understand why various optimization techniques work and how to select them. For example, to improve performance, if a workload is compute-bound, you should explore ways to optimize mathematical computations

and reduce FLOPs. If it is memory bandwidth-bound, you need to minimize unnecessary data movement.

Summary

This chapter began by showing you why LLM serving is crucial for business applications. Efficient LLM serving can improve customer experience, save costs, and even determine whether the business can be sustained and expanded.

Next, we turned to understanding AI accelerators, focusing on the key GPU specs of memory size, compute FLOPS, and memory bandwidth. You also learned why intra-node and inter-node GPU communication technology can be important for large models when a single GPU is not enough.

Additionally, we touched on the current state of other AI accelerators and how they are trying to overcome current limitations on GPU advancements, known as the “memory wall.”

With that understanding of the hardware side, we started to show you how an LLM runs on the hardware in both the model loading and model serving phases. We introduced arithmetic intensity as a key approximation technique in estimating model performance and identifying if it is bottlenecked by compute or memory bandwidth.

After some concrete examples with simplified calculations to illustrate the change in arithmetic intensity during Prefill and Decode, we concluded that LLM serving has both compute-bound phases and memory bandwidth-bound phases. When batch size is fixed at 1, LLM can be very memory bandwidth-bound, especially when Decode or the input sequence are not long enough. In this case, the GPU compute is not saturated enough to reach its full potential.

In this fast-evolving field, new techniques come out at an unprecedented pace. But the intuition you built from analysis will equip you to better understand them and adopt them if they fit your situation. Even if language models switch to a new architecture or you start working on vision models, your knowledge about hardware specs, how models run on hardware, and the general intuition will remain valuable.